

LLMs for AI – Assignment 3

AI Assignment 3 | Gour Krishna Dey | Roll - MT24035 | IIIT Delhi

Links - [Submission Drive Link](#)

Models Used

- **GPT-4.0** (by OpenAI)
- **Claude Sonnet 4.5** (by Anthropic)
- **Gemini 2.5 Pro** (by Google DeepMind)

Page Index

Page	Problem	Model Prompt Links
1	Rider Allocation	GPT-4.0 • Claude Sonnet 4.5 • Gemini 2.5 Pro
2	Container Loading	GPT-4.0 • Claude Sonnet 4.5 • Gemini 2.5 Pro
3	Kabaddi Game	GPT-4.0 • Claude Sonnet 4.5 • Gemini 2.5 Pro

Problem Set Overview

Rider Allocation (Page 4 Onwards)

Brief:

The goal of this problem is to **optimally assign delivery riders to customer orders** based on parameters like distance, time, and cost. The model must make real-time allocation decisions while minimizing total delivery time and ensuring fairness among riders.

Key Challenge:

Balancing workload distribution while minimizing travel distance.

Container Loading (Page 21 Onwards)

Brief:

This is a **combinatorial optimization problem** focused on maximizing space utilization when loading containers. The task involves deciding which items (of varying weights and sizes) to include so that space is efficiently used without exceeding capacity.

Key Challenge:

Handling the **3D bin-packing constraint** efficiently under limited computational resources.

Kabaddi Game Strategy Optimization (Page 37 Onwards)

Brief:

The objective here is to design an **AI strategy model for Kabaddi gameplay**, simulating decisions like raiding, defending, and team coordination. The model analyzes possible moves and predicts optimal strategies to maximize scoring chances.

Key Challenge:

Translating dynamic, multi-agent interactions into a decision framework understandable by an LLM.

1. Overview of Rider Allocation Problem

Prompt 1: Basic Proximity & Capacity (Easy)

Textproto

PROBLEM: You are a dispatcher. You have 2 idle riders and 2 new orders.

- Rider A is 1 km from Restaurant X.
- Rider B is 3 km from Restaurant X.
- Rider A is 4 km from Restaurant Y.
- Rider B is 2 km from Restaurant Y.

What is the optimal assignment to minimize total travel distance to the restaurants? Explain your reasoning step-by-step. Also add a small gist and rationale for your answer.

1. Response of GPT

Model / formulation (brief)

- Decision variables: assign $A \rightarrow X$ or $A \rightarrow Y$ (B gets the other).
- Objective: minimize total distance = $\text{dist}(A, \text{assigned}) + \text{dist}(B, \text{assigned})$.
- Constraints: each restaurant gets exactly one rider.

Method

Small problem — enumerate the two possible complete assignments and compare total distances (equivalent to solving the 2×2 assignment problem / Hungarian algorithm).

Work (calculation)

Distances:

- $A \rightarrow X = 1 \text{ km}$
- $A \rightarrow Y = 4 \text{ km}$
- $B \rightarrow X = 3 \text{ km}$
- $B \rightarrow Y = 2 \text{ km}$

Possible assignments:

1. $A \rightarrow X$ and $B \rightarrow Y \rightarrow \text{total} = 1 + 2 = \mathbf{3 \text{ km}}$
2. $A \rightarrow Y$ and $B \rightarrow X \rightarrow \text{total} = 4 + 3 = \mathbf{7 \text{ km}}$

Final answer

Optimal assignment: Rider A $\rightarrow$ Restaurant X, Rider B $\rightarrow$ Restaurant Y.
Minimum total travel distance = 3 km.

Gist & rationale

Gist: pick the pairing that yields the smaller sum of distances — here pairing A with X (1 km) and B with Y (2 km) beats the alternative (4+3).

Rationale: this choice pairs each rider to the nearer restaurant for that pairing, minimizing combined travel; with only two riders/orders you can check both assignments and choose the one with the smaller total.

2. Response of Claude

Problem Setup

We need to assign 2 riders to 2 restaurants to minimize total travel distance.

Given distances:

- Rider A → Restaurant X: 1 km
 - Rider B → Restaurant X: 3 km
 - Rider A → Restaurant Y: 4 km
 - Rider B → Restaurant Y: 2 km
-

Step-by-Step Solution

Step 1: Identify possible assignments

There are 2 possible assignment combinations:

1. **Assignment 1:** Rider A → Restaurant X, Rider B → Restaurant Y
 2. **Assignment 2:** Rider A → Restaurant Y, Rider B → Restaurant X
-

Step 2: Calculate total distance for each assignment

Assignment 1:

- Rider A travels to X: 1 km
- Rider B travels to Y: 2 km
- **Total distance = 1 + 2 = 3 km**

Assignment 2:

- Rider A travels to Y: 4 km
 - Rider B travels to X: 3 km
 - **Total distance = 4 + 3 = 7 km**
-

Step 3: Compare and select optimal assignment

Assignment 1 (3 km) < Assignment 2 (7 km)

FINAL ANSWER

Optimal Assignment:

- Rider A → Restaurant X
 - Rider B → Restaurant Y
 - Total Distance: 3 km
-

Gist

Assign each rider to the restaurant where they have a comparative advantage in distance savings.

Rationale

- Rider A has a **strong advantage** at Restaurant X (1 km vs 3 km - saves 2 km compared to Rider B)
- Rider B has a **moderate advantage** at Restaurant Y (2 km vs 4 km - saves 2 km compared to Rider A)
- By assigning riders to their relatively closer restaurants, we minimize the total system distance
- This follows the **Hungarian Algorithm principle** for optimal assignment problems - assign resources where opportunity cost is minimized

3. Response of Gemini

Here is the solution to the allocation problem.

1. Objective

To assign 2 riders (A and B) to 2 restaurants (X and Y) to **minimize the total combined travel distance** for both riders to reach their assigned restaurants.

2. Step-by-Step Reasoning

We can represent the distances (costs) in a simple table:

	Restaurant X	Restaurant Y

Rider A	1 km	4 km
Rider B	3 km	2 km

To find the minimum total distance, we must evaluate all possible 1-to-1 assignment scenarios:

- **Scenario 1:**
 - Rider A - Restaurant X (1 km)
 - Rider B - Restaurant Y (2 km)
 - **Total Distance = 1 km + 2 km = 3 km**
- **Scenario 2:**
 - Rider A - Restaurant Y (4 km)
 - Rider B - Restaurant X (3 km)
 - **Total Distance = 4 km + 3 km = 7 km**

Comparing the totals, Scenario 1 (3 km) is clearly lower than Scenario 2 (7 km).

3. Final Answer

The optimal assignment to minimize total travel distance is:

- **Rider A $\rightarrow$ Restaurant X**
- **Rider B $\rightarrow$ Restaurant Y**

Minimum Total Distance: 3 km

Gist & Rationale

- **Gist:** The best solution is assigning Rider A to Restaurant X and Rider B to Restaurant Y.
- **Rationale:** This allocation (1 km + 2 km) uses the two shortest individual routes available. The alternative (4 km + 3 km) uses the two *longest* routes. By pairing each rider with their closest option in this way, we achieve the lowest possible combined travel distance.

Prompt 2: Batched Order Dilemma (Moderate)

Textproto

PROBLEM: Rider Alex is 5 minutes from completing their first delivery. A new order appears. The restaurant is 2 minutes from Alex's current location, and the customer is 8 minutes from there. If assigned to Alex, this will delay his first delivery by 10 minutes. An idle rider, Sam, is 10 minutes from the new restaurant. The platform's policy is to avoid delaying existing orders by more than 8 minutes. The first customer's order is already 5 minutes behind the initial estimate. Should the new order be assigned to Alex or Sam? Justify your decision based on the constraints. Also add a small gist and rationale for your answer.

1. Response of GPT

Choose whether to give the new order to Alex (busy) or Sam (idle) while obeying the platform rule: **do not delay existing orders by more than 8 minutes.**

Model / formulation (brief)

- Decision: assign order to **Alex** or **Sam**.
- Constraint: existing order delay ≤ 8 minutes (platform policy).
- Objective (implicit): respect policy and avoid excessive customer delay.

Method

Check the consequence of each assignment against the constraint.

Work (calculation / reasoning)

- Current status: Alex is **5 minutes** away from completing his first delivery (that first customer is already **5 minutes behind** estimate).
- If Alex takes the new order, the problem statement says this **will delay his first delivery by 10 minutes**.
 - That 10-minute additional delay **exceeds** the platform limit of 8 minutes ($10 > 8$).
 - Even ignoring the platform limit, that makes the first order much later (already 5 min late + 10 min extra = 15 min late in total), which is bad for customer experience.
- If Sam (idle) takes the new order: Sam is **10 minutes** from the restaurant — slower for the new order, but **does not cause any extra delay to Alex's existing delivery**, so the platform constraint is respected.

Final answer

Assign the new order to Sam.

Reason: assigning to Alex would cause a **10-minute** delay to an existing order, which violates the platform policy (max allowed delay = 8 minutes). Assigning to Sam avoids violating the constraint.

Gist & rationale

Gist: **Never violate the platform's delay cap** — even if Alex is closer, his taking the job causes too large a delay to an in-progress delivery, so the idle rider Sam should take it.

Rationale: enforcing the delay limit protects the already-waiting customer and the platform's

service guarantees; it's better to accept a slightly longer fulfilment for the new order (Sam is slower) than to worsen an existing late delivery beyond the allowed threshold.

2. Response of Claude

Problem Setup

Decision: Assign new order to Alex (currently on delivery) or Sam (idle)?

Given Information:

- Alex: 5 min from completing current delivery
- New restaurant: 2 min from Alex's location
- Customer delivery: 8 min from restaurant
- Delay to Alex's first delivery if assigned: 10 minutes
- Alex's first customer is already 5 minutes delayed
- Sam (idle): 10 min from new restaurant
- **Policy constraint:** No delays > 8 minutes to existing orders

Step-by-Step Analysis

Step 1: Check policy constraint for assigning to Alex

- Alex's first customer current delay: 5 minutes
- Additional delay if new order assigned: 10 minutes
- **Total delay = 5 + 10 = 15 minutes**
- Policy limit: 8 minutes maximum delay
- **15 minutes > 8 minutes → VIOLATES POLICY ❌**

Step 2: Evaluate assigning to Sam

- Sam is idle (no existing commitments)
- No existing orders will be delayed
- Sam's travel time: 10 minutes to restaurant + 8 minutes to customer = 18 minutes total
- **No policy violations ✓**

Step 3: Compare outcomes

Option	Existing Order Delay	Policy Compliant?	New Order Completion Time
Alex	15 minutes	❌ NO	17 min (5+2+8+2)
Sam	0 minutes	✓ YES	18 min (10+8)

FINAL ANSWER

Assign the new order to Sam (idle rider)

Gist

Always prioritize policy compliance and existing customer commitments over marginal efficiency gains.

Rationale

1. **Policy Violation:** Assigning to Alex would cause a 15-minute delay (5 existing + 10 additional), exceeding the 8-minute policy limit
2. **Customer Trust:** The first customer is already delayed; further delays risk severe dissatisfaction and potential compensation claims
3. **Marginal Difference:** Sam completes the new order in 18 minutes vs ~17 minutes for Alex - only 1 minute slower
4. **Risk Management:** The small time savings (1 min) doesn't justify violating company policy and potentially losing a customer
5. **Operational Priority:** Protecting existing commitments maintains platform reliability and customer retention over optimizing individual delivery times

Decision principle: When constraints are violated, choose the compliant option even if slightly less efficient.

3. Response of Gemini

1. Objective

To assign the new order to either Alex or Sam while adhering to the platform's constraints, specifically the policy regarding delays to existing orders.

2. Step-by-Step Reasoning

The decision hinges entirely on the platform's constraints.

1. Analyze the Constraint: The platform policy states that an existing order **cannot be delayed by more than 8 minutes**.
2. Evaluate Rider Alex:
 - Alex is currently on a delivery (Customer 1).
 - If Alex is assigned the new order, it will delay his first delivery by **10 minutes**.
 - **Comparison:** 10 minutes (Alex's delay) > 8 minutes (Policy limit).
 - **Conclusion:** Assigning the order to Alex is a direct violation of the platform's policy.
3. Evaluate Rider Sam:
 - Sam is idle (not on a delivery).
 - Assigning the new order to Sam will cause **0 minutes** of delay to any *existing* order.
 - **Conclusion:** Assigning the order to Sam satisfies the constraint.

The fact that Alex is closer to the new restaurant (2 minutes) than Sam (10 minutes) is irrelevant because the assignment to Alex is forbidden by the 8-minute delay rule.

3. Final Answer

The new order must be assigned to **Sam**.

Gist & Rationale

- **Gist:** Sam gets the order.
- **Rationale:** Although Alex is much closer to the new restaurant, assigning him the order would delay his current delivery by **10 minutes**. This breaks the platform's strict rule of not delaying existing orders by more than **8 minutes**. Therefore, Sam must take the order as he is the only option that complies with the policy.

Prompt 3: Multi-Objective with Soft Constraints (Difficult)

Textproto

PROBLEM: You have 3 riders and 3 high-priority orders during a storm. Assignment options:

- Option 1: Total delivery time is 60 min, uses 2 riders, but 1 rider gets 2 orders.
- Option 2: Total delivery time is 45 min, uses 3 riders, but one order is 5 min late (small penalty).
- Option 3: Total delivery time is 75 min, uses 1 rider, all orders on time. Also add a small gist and rationale for your answer.

The platform wants to: 1) Minimize total time, 2) Avoid penalties, 3) Spread work fairly. Rank these options from best to worst, justifying your ranking by balancing the competing objectives. Also add a small gist and rationale for your answer.

1. Response of GPT

Rank the three storm-time assignment options (60 min/2 riders, 45 min/3 riders with 5-min lateness, 75 min/1 rider on-time) by balancing: (1) minimize total time, (2) avoid penalties, (3) spread work fairly.

Quick model / approach

Treat the three objectives as competing goals and score each option on:

- **Time** (lower is better),
- **Penalty** (none is best; a 5-min late order is a small penalty),
- **Fairness** (how well work is spread among riders).

Use a weighted combination that emphasizes time, then penalty, then fairness (example weights: time 50%, penalty 35%, fairness 15%) to produce an overall ranking. I'll show the intuition and result; if you want a different priority weighting, the ranking may change.

Scoring intuition (brief)

- **Option 1** — total time **60 min**, uses 2 riders (one rider has 2 orders)

- Time: medium-good.
 - Penalty: **none**.
 - Fairness: **moderately poor** (one rider overloaded).
- **Option 2** — total time **45 min**, uses 3 riders, **one order 5 min late**
 - Time: **best**.
 - Penalty: **small** (5 min late).
 - Fairness: **best** (work spread across 3 riders).
 - **Option 3** — total time **75 min**, uses 1 rider, all on time
 - Time: **worst**.
 - Penalty: **none**.
 - Fairness: **worst** (one rider gets everything).

Using the example weighting (time 0.50, penalty 0.35, fairness 0.15) and plausible normalized scores for each metric gives approximate combined scores:

- Option 2 $\approx$ **0.895**
- Option 1 $\approx$ **0.660**
- Option 3 $\approx$ **0.350**

Ranking (best → worst) with justification

1. **Option 2 (best) — 45 min, 3 riders, one order 5 min late.**
 - Why: It minimizes total system time (primary goal) and spreads load fairly (tertiary goal). The penalty (one order 5 min late) is small; because the platform's top priority is minimizing total time and storm conditions likely increase systemic risk, the time saving outweighs the modest penalty in most operational trade-offs.
2. **Option 1 (second) — 60 min, 2 riders, one rider has 2 orders.**
 - Why: No penalty (good for the second priority) but uses fewer riders and centralizes work (worse fairness) and has higher total time than Option 2. Option 1 is a conservative choice when penalties are very costly, but overall it's slower than Option 2.
3. **Option 3 (worst) — 75 min, 1 rider, all on time.**
 - Why: Though there are no penalties, it is the slowest and least fair (overloads a single rider) — it fails the platform's top objective (minimize total time) and is operationally risky during a storm.

Sensitivity note (short)

If the platform's policy treated any lateness as **unacceptable** (i.e., penalties are absolute and override time), **Option 1** or **3** would outrank Option 2. Conversely, if time savings are *extremely* valuable

(emergency / life-critical), Option 2 is clearly best. So the ranking above assumes the stated priorities are weighted roughly as “time > avoid small penalties > fairness.”

Final answer (boxed)

Best → Worst: Option 2 → Option 1 → Option 3.

Gist & rationale

Gist: favor the plan that gives the quickest overall completion (Option 2) unless the platform treats even small penalties as a firm no-go. Rationale: Option 2 best satisfies the primary objective (minimize total time) and also spreads work; its single small penalty is an acceptable trade-off given the big time savings in storm conditions.

2. Response of Claude

Problem Setup

Rank 3 assignment options for 3 high-priority orders during a storm, balancing multiple objectives.

Objectives (in order):

1. Minimize total delivery time
2. Avoid penalties
3. Spread work fairly

Options:

- **Option 1:** 60 min total, 2 riders, uneven distribution (1 rider gets 2 orders)
- **Option 2:** 45 min total, 3 riders, 1 order 5 min late (penalty)
- **Option 3:** 75 min total, 1 rider, all on time

Step-by-Step Analysis

Step 1: Evaluate each option against objectives

Option	Total Time	Penalty?	Work Distribution	Riders Used
Option 1	60 min	✓ None	⚠ Unfair (2-1-0)	2/3
Option 2	45 min	✗ Yes (1 late)	✓ Fair (1-1-1)	3/3
Option 3	75 min	✓ None	✗ Very unfair (3-0-0)	1/3

Step 2: Score by priority (weighted analysis)

Objective 1 - Minimize Total Time (Highest Priority):

- Option 2: **45 min** → Best ★★★★★
- Option 1: **60 min** → Moderate ★★★
- Option 3: **75 min** → Worst ★

Objective 2 - Avoid Penalties (Second Priority):

- Option 1: No penalties → Best ★★★★★
- Option 3: No penalties → Best ★★★★★
- Option 2: 1 late order (small penalty) → Acceptable ★★

Objective 3 - Fair Work Distribution (Third Priority):

- Option 2: All 3 riders used equally → Best ★★★★★
 - Option 1: 2 riders, uneven load → Moderate ★★★
 - Option 3: Only 1 rider works → Worst ★
-

Step 3: Consider storm context

During a storm:

- **Speed is critical** (customers waiting longer, dangerous conditions)
 - **Rider safety** matters (spreading work reduces individual exposure time)
 - **Small penalties** may be acceptable given extreme conditions
 - Getting orders delivered quickly reduces overall risk
-

FINAL RANKING

🏆 **BEST: Option 2** (45 min, 3 riders, 1 small penalty)

🥈 **SECOND: Option 1** (60 min, 2 riders, no penalty)

🥉 **WORST: Option 3** (75 min, 1 rider, no penalty)

Gist

In emergency conditions, prioritize speed and resource utilization over avoiding minor penalties.

Rationale

Why Option 2 wins:

1. **25% faster than Option 1** (45 vs 60 min) - significant customer experience improvement

2. **40% faster than Option 3** (45 vs 75 min) - drastically better during storm
3. **Fair workload distribution** - no single rider overburdened in dangerous conditions
4. **Small penalty is acceptable** - 5 minutes late during a storm is reasonable, customers expect delays
5. **Operational efficiency** - uses all available resources during high-demand period
6. **Risk mitigation** - spreading work means no rider is overexposed to storm conditions

Why Option 1 is second:

- Penalty-free but 33% slower than Option 2
- Unfair distribution creates rider resentment (one does double work)
- Leaves one rider idle during peak demand

Why Option 3 is worst:

- Slowest option by far (75 min)
- Terrible for customer experience during emergency
- Unfair to the single working rider (safety risk in storm)
- Inefficient use of available workforce
- Two riders sit idle while one struggles

Decision Principle: When objectives conflict, prioritize by hierarchy. Total time (Objective 1) saves 15-30 minutes, which outweighs a 5-minute penalty (Objective 2). In extreme conditions, accept minor penalties to optimize primary objectives.

3. Response of Gemini

Here is the solution to the allocation problem.

1. Objective

To rank the three assignment options from best to worst by balancing the platform's competing priorities:

1. **Minimize total time.** (Highest Priority)
 2. **Avoid penalties.** (Medium Priority)
 3. **Spread work fairly.** (Lowest Priority)
-

2. Evaluation and Ranking

We will evaluate each option against the three priorities.

- **Option 1:** (60 min, 2 riders, 0 penalties)
 - Time (P1): 60 min (Ranks 2nd)
 - Penalties (P2): 0 (Pass)
 - Fairness (P3): Poor (1 rider is overloaded, 1 is idle)
- **Option 2:** (45 min, 3 riders, 1 small penalty)

- Time (P1): 45 min (Ranks 1st - Best)
 - Penalties (P2): 1 (Fail)
 - Fairness (P3): Good (All 3 riders are used)
 - **Option 3:** (75 min, 1 rider, 0 penalties)
 - Time (P1): 75 min (Ranks 3rd - Worst)
 - Penalties (P2): 0 (Pass)
 - Fairness (P3): Very Poor (1 rider is heavily overloaded, 2 are idle)
-

3. Final Answer (Ranking)

Based on the prioritized objectives, the ranking is:

1. Best: Option 2

- **Justification:** This option is the only one that achieves the **highest priority goal** (Minimize total time), being 15 minutes faster than the next best. It also achieves the third priority (Fairness). While it fails the second priority, the problem specifies the penalty is "small," making it an acceptable trade-off for a significant win on the main objective.

2. Middle: Option 1

- **Justification:** This option is a compromise. It successfully avoids penalties (P2) but fails on fairness (P3) and is significantly slower than Option 2 (failing P1). It is "safer" than Option 2 but less efficient.

3. Worst: Option 3

- **Justification:** This option fails spectacularly on the two most important goals. It is the **slowest** (P1) and the **most unfair** (P3). Its only advantage (avoiding penalties) does not compensate for its extreme inefficiency.
-

Gist & Rationale

- **Gist:** The best choice is **Option 2 (45 min)**, followed by Option 1 (60 min), with Option 3 (75 min) being the worst.
- **Rationale:** The platform's top priority is minimizing total time, and Option 2 is the clear winner at 45 minutes. Incurring a "small penalty" is a reasonable price to pay for being 15 minutes (25%) faster *and* achieving fair work distribution (P1 and P3). Option 3 is the worst as it is both the slowest and the most unfair.

Prompt 4: Dynamic Re-Routing (Very Difficult)

Textproto

PROBLEM: Rider Ben is on a route to deliver Order 1 (10 min away) then Order 2 (another 5 min). A traffic jam will delay Order 1 by 15 minutes. You have two choices:

A) Stick to the plan: Order 1 delayed 15 min, Order 2 delayed 15 min.

B) Re-route: Ben delivers Order 2 first (now 12 min away), then Order 1 (another 8 min). This makes Order 1 20 min late but Order 2 on time.

Which option minimizes total lateness? Calculate the total lateness (sum of minutes late for all orders) for each option and choose the best one. Also add a small gist and rationale for your answer.

1. Response of GPT

Choose between sticking with the current route (both orders hit the traffic jam) or re-routing so Order 2 is served first — which option gives smaller total lateness?

Model / formulation (brief)

- Two orders: Order 1 and Order 2.
- Lateness = minutes past the promised/due time for each order.
- Objective: minimize total lateness = Lateness(Order1) + Lateness(Order2).

Method

Use the given consequences for each option and compute the sum of lateness for both orders; compare totals.

Work (calculations)

Option A (stick to plan):

- Order 1 lateness = 15 minutes.
- Order 2 lateness = 15 minutes.
- Total lateness = 15 + 15 = 30 minutes.

Calculation shown digit-by-digit:

15

+15

30

Option B (re-route: deliver Order 2 first):

- Order 1 lateness = 20 minutes.
- Order 2 lateness = 0 minutes (on time).

- Total lateness = $20 + 0 = 20$ minutes.

Digit-by-digit:

20

- 0

20

Final answer

Choose Option B (re-route).

Total lateness: Option A = **30 minutes**; Option B = **20 minutes**.

Option B minimizes total lateness.

Gist & rationale

Gist: even though re-routing makes Order 1 worse (20 vs 15), it eliminates Order 2's 15-minute delay — the net effect is a 10-minute reduction in total lateness.

Rationale: when the objective is to minimize the sum of lateness across orders, prioritizing the order you can make on time (Order 2) and accepting a larger delay for the other order produces the smaller aggregate penalty.

2. Response of Claude

Problem Setup

Ben has 2 orders. A traffic jam affects the current route. Choose between sticking to plan or re-routing to minimize total lateness.

Current Plan:

- Order 1: 10 min away
- Order 2: 5 min after Order 1 (15 min total)
- Traffic jam delays: +15 min

Re-route Option:

- Order 2 first: 12 min away
- Order 1 after: 8 min more (20 min total)

Step-by-Step Calculation

Option A: Stick to the Plan (with traffic jam)

Order 1:

- Original ETA: 10 min

- With traffic delay: $10 + 15 = 25$ min
- **Lateness: 15 minutes**

Order 2:

- Original ETA: 15 min ($10 + 5$)
- With traffic delay: $15 + 15 = 30$ min
- **Lateness: 15 minutes**

Total Lateness (Option A) = $15 + 15 = 30$ minutes

Option B: Re-route (Order 2 first, then Order 1)

Order 2:

- New route time: 12 min
- Original ETA: 15 min
- Actual delivery: 12 min
- **Lateness: 0 minutes** (actually 3 min early, but counts as 0)

Order 1:

- New route time: $12 + 8 = 20$ min
- Original ETA: 10 min
- Actual delivery: 20 min
- **Lateness: 10 minutes**

Total Lateness (Option B) = $0 + 10 = 10$ minutes

Comparison

Option	Order 1 Lateness	Order 2 Lateness	Total Lateness
A (Stick to plan)	15 min	15 min	30 min
B (Re-route)	10 min	0 min	10 min

Savings: $30 - 10 = 20$ minutes of total lateness reduced

FINAL ANSWER

Choose Option B: Re-route to deliver Order 2 first, then Order 1

- **Total Lateness: 10 minutes** (vs 30 minutes for Option A)
- **67% reduction in total customer wait time**

Gist

When faced with unavoidable delays, re-routing to save at least one order significantly reduces total customer dissatisfaction.

Rationale

1. **Total Lateness Optimization:** Option B reduces cumulative lateness by 20 minutes (30 vs 10), a 67% improvement
2. **Customer Impact Distribution:**
 - Option A: Both customers suffer equally (15 min each)
 - Option B: One customer unaffected, one experiences manageable delay (10 min vs 15 min)
3. **Service Recovery:** Delivering one order on time maintains partial service quality rather than failing both customers
4. **Psychological Factor:** One satisfied customer (on-time delivery) is better than two moderately dissatisfied customers
5. **Complaint Reduction:** Option B likely generates 1 complaint vs 2 complaints in Option A
6. **Smart Adaptation:** Re-routing demonstrates proactive problem-solving rather than passive acceptance of delays

Decision Principle: When disruptions are unavoidable, minimize aggregate impact by salvaging what you can. Delivering 50% on-time with controlled damage is superior to universal moderate failure.

2. Response of Gemini

Here is the solution to the allocation problem.

1. Objective

To minimize the **total combined lateness** (the sum of the delay minutes for both Order 1 and Order 2).

2. Step-by-Step Reasoning

We must calculate the total lateness for both scenarios.

- **Option A: Stick to the plan**
 - Order 1 Lateness: 15 minutes
 - Order 2 Lateness: 15 minutes (The 15-minute delay on Order 1 cascades, making Order 2 also 15 minutes late)
 - **Total Lateness = 15 + 15 = 30 minutes**
- **Option B: Re-route**
 - Order 1 Lateness: 20 minutes
 - Order 2 Lateness: 0 minutes (Delivered on time)
 - **Total Lateness = 20 + 0 = 20 minutes**

Comparing the two totals, Option B (20 minutes) results in less total lateness than Option A (30 minutes).

3. Final Answer

The best choice is **Option B (Re-route)**.

Minimum Total Lateness: 20 minutes

Gist & Rationale

- **Gist:** Option B is the better choice.
- **Rationale:** While Option B makes Order 1 significantly later (20 min vs 15 min), it completely saves Order 2 from *any* delay. This "sacrifice" results in a lower overall system delay (20 total minutes) compared to Option A, where the delay cascades and negatively impacts both customers (30 total minutes).

2. Overview of Container Loading Problem

Prompt 1: Easy (Single Box Type, Volume Maximization)

Textproto

A shipping container has internal dimensions of **10** meters in length, **5** meters in width, and **4** meters in height. You have **40** identical boxes to pack. Each box is a cube with **2-meter** long sides.

Using the Container Loading Problem approach, what is the maximum number of these boxes that can be packed into the container? Explain how you would arrange them to achieve this number. Calculate both the total volume of the packed boxes and the utilization of the container's volume.

1. Response of GPT

Solution (brief reasoning + final answer)

Container internal size = $10 \times 5 \times 4$ m $\rightarrow$ volume = $10 \cdot 5 \cdot 4 = 200$ m³.

Each box is a cube with side 2 m $\rightarrow$ box volume = $2^3 = 8$ m³.

Step-by-step packing count (axis-by-axis, integer fit):

- Along length (10 m): $\lfloor 10/2 \rfloor = 5$ boxes.
- Along width (5 m): $\lfloor 5/2 \rfloor = 2$ boxes.
- Along height (4 m): $\lfloor 4/2 \rfloor = 2$ boxes.

Total boxes that fit = $5 \times 2 \times 2 = 20$ boxes.

You have 40 boxes available, but only **20** can be loaded given the container dimensions (boxes are rigid cubes; rotations don't change size).

Volume calculations:

- Volume per box = $2 \times 2 \times 2 = 8 \text{ m}^3$.
- Total volume of packed boxes = $20 \times 8 = 160 \text{ m}^3$.
- Container volume = $10 \times 5 \times 4 = 200 \text{ m}^3$.
- Volume utilization = $160/200 = 0.8 = 80\%$.
- Unused volume = $200 - 160 = 40 \text{ m}^3$.

Arrangement to achieve this:

Place boxes in a regular grid of 5 (length) $\times$ 2 (width) $\times$ 2 (height). That is, 5 boxes end-to-end along the 10 m length, 2 across the 5 m width, and 2 stacked in the 4 m height.

Final answer (concise)

- **Maximum boxes that can be packed: 20** (of the 40 available).
- **Arrangement:** grid $5 \times 2 \times 2$ (length $\times$ width $\times$ height).
- **Total packed boxes volume: 160 m³.**
- **Container utilization: 80%** (unused volume 40 m³).

2. Response of Claude

Solution Approach

Step 1: Calculate volumes

- Container volume = $10 \times 5 \times 4 = \mathbf{200 \text{ cubic meters}}$
- Each box volume = $2 \times 2 \times 2 = \mathbf{8 \text{ cubic meters}}$
- Total available boxes = 40

Step 2: Determine fitting along each dimension

- Length (10m): $10 \div 2 = \mathbf{5 \text{ boxes}}$
- Width (5m): $5 \div 2 = \mathbf{2 \text{ boxes}}$ (with 1m remaining)
- Height (4m): $4 \div 2 = \mathbf{2 \text{ boxes}}$

Step 3: Calculate maximum capacity Maximum boxes that can fit = $5 \times 2 \times 2 = \mathbf{10 \text{ boxes}}$

Step 4: Arrangement

- **First layer (bottom):** 5 boxes along length $\times$ 2 boxes along width = 10 boxes
- **Second layer (top):** 5 boxes along length $\times$ 2 boxes along width = 10 boxes
- **Total arrangement:** 2 layers of 10 boxes each = 20 boxes

However, we only have 10 boxes that fit perfectly within the container dimensions.

Step 5: Calculate utilization

- Volume of packed boxes = 10 boxes $\times$ 8 m³ = **80 cubic meters**
- Container utilization = $(80 \div 200) \times 100 = 40\%$

Final Answer

Maximum number of boxes: 10 boxes (limited by container dimensions, not by availability)

Arrangement: 5 boxes (length) $\times$ 2 boxes (width) $\times$ 1 layer (height) = 10 boxes, OR 5 (length) $\times$ 1 (width) $\times$ 2 (height) = 10 boxes

Total volume of packed boxes: 80 cubic meters

Container utilization: 40%

3. Response of Gemini

Here is the solution based on the Container Loading Problem (CLP) approach.

Solution Brief

The Container Loading Problem requires us to determine the physical fit of objects, not just compare total volumes.

1. **Container Volume:** $10\text{m} \times 5\text{m} \times 4\text{m} = 200 \text{ m}^3$
2. **Box Volume:** $2\text{m} \times 2\text{m} \times 2\text{m} = 8 \text{ m}^3$
3. **Theoretical Max (Volume Only):** $200 \text{ m}^3 / 8 \text{ m}^3 = 25$ boxes.
4. **Actual Fit (CLP):** We must check how many 2m boxes fit along each dimension.
 - Along the 10m length: $10\text{m} / 2\text{m} = 5$ boxes
 - Along the 5m width: $5\text{m} / 2\text{m} = 2.5$. We can only fit **2 boxes**, leaving 1m of wasted space.
 - Along the 4m height: $4\text{m} / 2\text{m} = 2$ boxes
5. **Maximum Packed Boxes:** The maximum number that can be physically arranged is $5 \times 2 = 20$ boxes. Since we have 40 boxes, the container space is the limiting factor.
6. **Total Packed Volume:** $20 \text{ boxes} \times 8 \text{ m}^3/\text{box} = 160 \text{ m}^3$.
7. **Volume Utilization:** $(\text{Packed Volume} / \text{Container Volume}) \times 100 = (160 \text{ m}^3 / 200 \text{ m}^3) \times 100 = 80\%$.

Final Answer

- **Maximum Boxes:** 20 boxes (Note: This is less than the theoretical 25 because the 5-meter width only allows 2 boxes to be placed side-by-side).
- **Arrangement:** A $5 \times 2 \times 2$ arrangement. This means packing 5 boxes along the 10m length, 2 boxes along the 5m width (using 4m and wasting 1m), and 2 boxes high along the 4m height.
- **Total Packed Box Volume:** 160 m³
- **Container Volume Utilization:** 80%

Prompt 2: Medium (Multiple Box Types, Orientation, and Basic Constraints)

Textproto

Consider a container with dimensions 12 ft (L) x 8 ft (W) x 10 ft (H). You have three types of boxes to load:

Type A: 20 boxes, each 4 ft x 4 ft x 4 ft.

Type B: 15 boxes, each 6 ft x 4 ft x 2 ft.

Type C: 10 boxes, each 3 ft x 3 ft x 3 ft.

All boxes can be rotated. Your goal is to maximize the total number of boxes loaded. Propose a high-level packing strategy. Which boxes will you load first and in what orientation? What is the theoretical maximum number of boxes you could fit if volume was the only constraint, and why might this not be achievable in practice?

Response of GPT

Container & box volumes / quick facts

- Container: 12 ft x 8 ft x 10 ft → container volume = $12 \times 8 \times 10 = 960$ ft³.
- Type A (cube 4x4x4): volume = $4^3 = 64$ ft³. available 20.
- Type B (6x4x2): volume = $6 \times 4 \times 2 = 48$ ft³. available 15.
- Type C (cube 3x3x3): volume = $3^3 = 27$ ft³. available 10.
- Total volume of all boxes = $20 \cdot 64 + 15 \cdot 48 + 10 \cdot 27 = 1280 + 720 + 270 = 2270$ ft³ (>> container).

High-level packing strategy (goal = maximize number of boxes)

1. **Load smallest boxes first (Type C)** — they give the best “boxes per ft³” so maximize count.
2. **Then load medium boxes (Type B)** — they are next-best for count-per-volume.
3. **Finally place large boxes (Type A)** in remaining spaces if room allows.

Rationale: to maximize *count*, use as many small items as possible before using larger-volume items.

Suggested orientations (to help actual packing)

- **Type C (3x3x3):** cube — orientation irrelevant. Best to orient so they form a 3D grid.
 - Grid capacity if only C: along L: $12/3=4$, W: $8/3=2$ (remainder 2 ft), H: $10/3=3$ (uses 9 ft). Pure grid capacity =

$4 \times 2 \times 3 = 24$ positions (you only have 10).

- **Type B (6×4×2):** choose orientation **6 ft (L) × 4 ft (W) × 2 ft (H)** so you can stack many thin layers.
 - Capacity per that orientation: L: $12/6=2$, W: $8/4=2$, H: $10/2=5$ → $2 \times 2 \times 5 = 20$ positions (only 15 available).
- **Type A (4×4×4):** cube — orientation irrelevant.
 - Grid capacity: L: $12/4=3$, W: $8/4=2$, H: $10/4=2$ (8 ft used) → $3 \times 2 \times 2 = 12$ positions.

Practical tip: place C boxes into a regular $3 \times 2 \times 3$ block(s) filling corners/shelves, then place B blocks (6×4×2) in remaining rectangular bays, and finally try to fit As into rectangular leftovers.

Theoretical maximum by volume-only (ignore shape/fit)

To maximize **count** under pure volume constraint, choose smallest-volume boxes first (we're limited by availability).

- Take all **10 Type C** → uses $10 \times 27 = 270$ ft³. Remaining capacity = $960 - 270 = 690$ ft³.
- Fill remaining capacity with Type B (48 ft³ each): $\lfloor 690/48 \rfloor = 14$ boxes → uses $14 \times 48 = 672$ ft³. Remaining capacity = $690 - 672 = 18$ ft³ (too small for any box).
- Total boxes by volume-only = **10 (C) + 14 (B) = 24 boxes**.
(You cannot use Type A instead to increase the *count* because A has larger volume; replacing B by A would not increase count.)

So **theoretical max (volume-only) = 24 boxes**.

Why the volume-only maximum may NOT be achievable

- **Geometric constraints / discreteness:** boxes are rigid; you must place them on an integer grid along each axis. Even if total volume fits, the container's dimensions might not permit the required packing pattern.
- **Fragmentation / unusable gaps:** leftover spaces may be too narrow or oddly shaped for any remaining box (e.g., leftover 2 ft strips), producing wasted volume.

- **Orientation limits & interference:** mixing different box sizes can create pockets that can't be filled by available box shapes.
Boundary mismatches: e.g., Type C uses 3 ft — width 8 ft leaves a 2 ft strip unusable; height 10 ft leaves 1 ft when stacking 3-high (3×3×3). Those leftover strips can prevent fitting as many B or A boxes as volume-only suggests.
- **Availability limitations:** volume-only used inventory counts; in practice you might not be able to place the 14 B boxes in contiguous positions required.

Because of those issues, the actual physically-achievable maximum will often be **less than or equal to** the volume-only number; a realistic mixed-grid packing plan and small optimization/search (or manual arrangement) is needed to find the true max arrangement (e.g., you might reach 23 or 24 in a lucky geometry but often will be lower).

Short final answer

- **Packing priority:** Type C first (smallest), then Type B, then Type A.
- **Recommended orientations:** B as **6×4×2** (height = 2) so you can stack 5 layers; A and C are cubes so orientation doesn't matter.
- **Volume-only theoretical maximum boxes: 24 boxes** (10 Type C + 14 Type B).
- **Why not guaranteed:** geometric/fit constraints, leftover gaps, and discrete placement may prevent actually achieving 24 even though volumes add up.

Response of Claude

Solution Approach

Step 1: Calculate volumes

- Container volume = $12 \times 8 \times 10 = 960$ **cubic feet**
- Type A: $4 \times 4 \times 4 = 64$ **cu ft each** (20 available)
- Type B: $6 \times 4 \times 2 = 48$ **cu ft each** (15 available)
- Type C: $3 \times 3 \times 3 = 27$ **cu ft each** (10 available)

Step 2: Theoretical maximum (volume-only constraint) If we could perfectly pack based on volume alone:

- Total available volume = $(20 \times 64) + (15 \times 48) + (10 \times 27) = 1,280 + 720 + 270 = 2,270$ **cu ft**
- Container can hold: 960 cu ft
- Theoretical boxes to fill 960 cu ft:
 - All Type C: $960 \div 27 = 35.5$ **boxes** (but only 10 available)
 - All Type B: $960 \div 48 = 20$ **boxes** (but only 15 available)
 - All Type A: $960 \div 64 = 15$ **boxes** (20 available)

Best theoretical mix: All 10 Type C (270 cu ft) + All 15 Type B (720 cu ft) = **25 boxes using 990 cu ft** (exceeds by 30 cu ft, so adjust)

Step 3: High-Level Packing Strategy

Priority Order: Type C → Type B → Type A

Reasoning:

- Type C has best volume efficiency (smallest, easiest to fill gaps)
- Type B is medium-sized with flexible orientation
- Type A is largest (least flexible, hardest to optimize)

Step 4: Proposed Loading Plan

Phase 1 - Load Type B first (counter-intuitively better):

- Orient Type B: 6ft(L) × 4ft(W) × 2ft(H)
- Bottom layer: 2 boxes along length (12ft) × 2 along width (8ft) × 5 layers high (10ft) = **20 boxes** - but we only have 15
- Actual: Load all **15 Type B boxes** in orientation 6×4×2
- Arrangement: 2(L) × 2(W) × 3 layers = 12 boxes, plus 3 more in partial 4th layer
- Space used: approximately 12ft × 8ft × 8ft

Phase 2 - Load Type C in remaining space:

- Remaining height: 2 ft (top layer)
- Can fit: 4 boxes (12ft÷3) × 2 boxes (8ft÷3 = 2.67, so 2) × 0 layers vertically (only 2ft available, need 3ft)
- **0 Type C boxes fit in top space**
- Try gaps: Potentially 2-3 Type C boxes in small gaps

Phase 3 - Type A:

- Too large to fit efficiently with remaining space

Step 5: Why theoretical maximum is not achievable

Reasons:

1. **Geometric constraints** - Boxes have fixed dimensions that don't divide evenly into container dimensions
2. **Wasted space** - Gaps and voids are inevitable (e.g., $8 \div 3 = 2.67$ leaves unusable space)
3. **Rotation limitations** - Not all orientations work for available spaces
4. **3D Tetris problem** - Unlike 2D, you can't easily fill all gaps in 3D space
5. **Load order dependencies** - Early choices lock out later optimal placements

Final Answer

High-level packing strategy:

1. **Load Type B first** (orientation: 6×4×2) - moderate size, good space filling
2. **Load Type C second** - fills smaller gaps

3. **Load Type A last** - only if space permits

Recommended loading:

- **12-15 Type B boxes + 2-4 Type C boxes = ~16-18 boxes total**

Theoretical maximum by volume only:

- **~25 boxes** (all Type C + most Type B)

Why not achievable:

- Geometric packing constraints create **15-30% wasted space**
- Actual achievable: **16-20 boxes** (~40-50% fewer than volume suggests)
- Container utilization: **60-75%** realistic vs. 100% theoretical

Response of Gemini

High-Level Packing Strategy

The best strategy is to load the largest, most dimensionally restrictive boxes first, as they create the most stable base and the cleanest remaining sub-spaces. This is a common heuristic for the Container Loading Problem.

1. **Load Type A (Largest Volume) First:**

- **Boxes:** Type A (4ft x 4ft x 4ft).
- **Orientation:** 4ft along the 12ft length (fits 3), 4ft along the 8ft width (fits 2), and 4ft along the 10ft height (fits 2).
- **Action:** Load a block of $3 \times 2 \times 2 = \text{12 Type A boxes}$.
- **Remaining Space:** This uses a $12\text{ ft} \times 8\text{ ft} \times 8\text{ ft}$ volume, leaving a single, clean "shelf" of $12\text{ ft} \times 8\text{ ft} \times 2\text{ ft}$ at the top.

2. **Load Type B (Fits Remaining Space) Second:**

- **Boxes:** Type B (6ft x 4ft x 2ft).
- **Orientation:** Rotate the box to $6\text{ ft (L)} \times 4\text{ ft (W)} \times 2\text{ ft (H)}$. This 2ft height fits the remaining 2ft "shelf" perfectly.
- **Action:** Fit 2 boxes along the 12ft length ($2 \times 6\text{ ft} = 12\text{ ft}$) and 2 boxes along the 8ft width ($2 \times 4\text{ ft} = 8\text{ ft}$). This loads $2 \times 2 = \text{4 Type B boxes}$.
- **Remaining Space:** None. The container is now full.

3. **Load Type C:**

- There is no space left for Type C boxes.

Final Packed Count: 12 (Type A) + 4 (Type B) = **16 boxes**.

Theoretical Maximum (Volume-Based)

To find the *theoretical* maximum *number* of boxes, we ignore physical fit and only consider volume. We prioritize loading the smallest boxes first, as this maximizes the item count for a given volume.

- **Container Volume:** $12\text{ ft} \times 8\text{ ft} \times 10\text{ ft} = 960\text{ cu ft}$
- **Box Volumes:**
 - Type C: $3\text{ ft} \times 3\text{ ft} \times 3\text{ ft} = 27\text{ cu ft}$
 - Type B: $6\text{ ft} \times 4\text{ ft} \times 2\text{ ft} = 48\text{ cu ft}$
 - Type A: $4\text{ ft} \times 4\text{ ft} \times 4\text{ ft} = 64\text{ cu ft}$
- 1. **Load all 10 Type C (smallest):** $10 \times 27\text{ cu ft} = 270\text{ cu ft}$
 - *Volume remaining:* $960 - 270 = 690\text{ cu ft}$
- 2. **Load Type B (next smallest):** $690\text{ cu ft} / 48\text{ cu ft} = 14.375$. We can theoretically fit **14 Type B boxes**.
 - *Volume used by Type B:* $14 \times 48 = 672\text{ cu ft}$
- 3. **Total:** $10\text{ (Type C)} + 14\text{ (Type B)} = \text{24 boxes}$
 - *Total volume used:* $270 + 672 = 942\text{ cu ft}$ (This is 98.1% utilization).

Theoretical Maximum: 24 boxes

Why This Is Not Achievable

The theoretical maximum of 24 boxes is not achievable in practice because of **3D spatial constraints**.

The volume-based calculation treats the boxes like a liquid—as if you could crush them up and pour them into the container. In reality, rigid boxes must fit their specific dimensions.

- **Dimensional Mismatch:** The container's 8ft width cannot be perfectly filled by 3ft boxes ($3+3=6$, leaving 2ft of wasted space).
- **Wasted Space (Void):** Loading boxes creates unavoidable gaps and "voids" where the remaining space is too small or the wrong shape to fit any of the remaining boxes, even if the total volume of these voids is large.
- **Fragmentation:** As you load, the remaining space becomes fragmented into complex, non-rectangular shapes, making it impossible to pack efficiently.

Prompt 3: Hard (Complex Constraints, Fragility, and Weight Distribution)

Textproto

You are loading a **20-foot** container (internal dimensions: **19.5** ft L x **7.5** ft W x **7.5** ft H). You must load the following items:

Pallets: **5** heavy, non-stackable metal pallets. Each is **4** ft x **4** ft x **0.5** ft and weighs **200** lbs.

Crates: **12** sturdy wooden crates. Each is **3** ft x **2** ft x **2** ft and can be stacked.

Boxes: **25** fragile cardboard boxes. Each is **2** ft x **1.5** ft x **1** ft. They can be stacked, but only on top of each other (nothing heavier can be placed on them).

Constraints:

The total weight must be evenly distributed; the heavy pallets cannot all be placed on one side.

The fragile boxes must be loaded last and placed on top of the stack to prevent crushing.

The container must be stable for transport; a lower center of gravity is better.

Devise a loading plan. Describe the sequence of loading (what goes in first, what goes in last) and the general arrangement of items to satisfy all constraints. Do not provide exact coordinates, but describe the logic of your packing strategy.

Response of GPT

Quick feasibility checks (numbers you can use mentally)

- Container interior: **19.5 ft L × 7.5 ft W × 7.5 ft H**. Floor area = **146.25 ft²**; usable height = **7.5 ft**.
- Pallet footprint: **4×4 = 16 ft²** each; 5 pallets → **80 ft²** on the floor (non-stackable).
- Crate footprint: **3×2 = 6 ft²** each; 12 crates → 72 ft² if all on floor, but crates are stackable so you can free floor area by stacking.
- Fragile boxes footprint: **2×1.5 = 3 ft²** each; 25 boxes → 75 ft² if all on floor, but they are to be stacked on top only — they will be placed on top of crates or crate stacks.

Height stacking consideration (working plan below): stacking crates **2 high** = 4 ft; stacking boxes up to **3 high** on top = 3 ft → total top-of-stack ≈ 7 ft < 7.5 ft; this keeps everything within the container height with a margin.

Packing philosophy (high level)

1. **Keep the heaviest items low and spread them out** to achieve even weight distribution across the length and width.
2. **Use stackable crates to create elevated, stable platforms** for the fragile boxes — this frees floor space and keeps fragile items above heavier items.
3. **Load fragile boxes last**, always on top of crate stacks only, and never under or beside the heavy pallets.
4. **Aim for a low, centered center-of-gravity**: heavy pallets on the floor, crates mostly in the middle and rear thirds, fragile boxes on top but centrally located.
5. **Secure and brace** everything (straps, blocking, dunnage) to avoid shifting.

Loading sequence (step-by-step, logical order)

1. Pre-check & plan the zones
 - Mentally divide the container lengthwise into fore / mid / aft thirds and use the width to make left / center / right lanes. The goal is to balance weight both left-right and fore-aft.
2. Load the 5 heavy, non-stackable pallets first (on the floor)
 - Place them directly on the container floor because they cannot be stacked.
 - **Distribution rule**: don't cluster them — spread them left/right and fore/aft. Example logic: two pallets on the left side (one forward, one aft), two on the right side (one forward, one aft), and the fifth on the centerline near the mid-length.
 - This arrangement evens out point loads on the floor, prevents one-side overload, and keeps the heavy mass low.

3. Place crates next and form stackable platforms

- Use the crates to fill floor gaps around and between pallets while keeping the crates' stacks primarily **centered** between the pallets (so boxes above are supported by a central, low mass).
- **Stacking strategy for crates:** stack most crates **2 high** ($2 \times 2 \text{ ft} = 4 \text{ ft}$) to free floor space – with 12 crates you can make **6 stacks of 2** (or a mixture of 2-high and 1-high depending on space).
- Place some single-level crates in narrow leftover bays if needed. Keep crate stacks compact and square/rectangular so they form stable platforms for boxes.

4. Stow any additional crates to help balance weight

- If pallet placement left one side lighter, use crate stacks to compensate: move some stacks slightly toward the lighter side to balance the load. Remember crates are much lighter than pallets, so fine-tune placement to achieve even distribution.

5. Load the fragile boxes last – on top of crate stacks only

- Place the 25 fragile boxes **only** on top of crate stacks (never on pallets, never under heavier items).
- **Stack height for boxes:** keep to a safe stack (e.g., up to **3 boxes high** = 3 ft) so a crate stack (4 ft) + 3 boxes (3 ft) = ~7 ft total – within the container height with margin.
- Spread fragile-box stacks across several crate platforms rather than piling them all on one stack (this reduces point load and risk of collapse and helps balance).
- Keep fragile-box stacks toward the centerline and mid-length where vibrations are lower than at the doors and to maintain stability.

6. Final balancing, blocking, and securing

- After loading, check even fore-aft axle loading (visually and by weight if possible). Adjust crate placement left-right and fore-aft to even out.
- Use heavy-duty straps and edge protectors across pallets and crate stacks. Use blocking/dunnage at container ends or between pallets/crates to prevent movement in transit.
- Place lighter loose items (if any) in remaining small voids near the roofline but not on top of fragile boxes.
- Ensure pallet corners and crate corners are braced so that top box stacks are on a rigid surface.

How this plan satisfies your constraints

- **Heavy pallets evenly distributed:** Pallets placed on floor in left-right / fore-aft pattern avoids one-side concentration.
- **Fragile boxes loaded last & only on top:** Boxes are placed only on crate stacks and added after all heavy items are fixed. Nothing heavier is placed above fragile boxes.
- **Lower center of gravity / stability:** Heavy pallets stay on the floor; crates (medium mass) below most of the boxes; fragile boxes are on top but light, keeping the overall center of gravity low.
- **Stacking safety & height limits:** Crates 2-high + boxes 3-high keep stacks $\leq \sim 7$ ft (< 7.5 ft).
- **Transport stability:** Even distribution and strapping/bracing minimize load-shift risk and reduce roll/tilt tendencies.

Practical tips & operational cautions

- If crates carry different contents or weights, put heavier crates lower (single level) and lighter crates on top.
- Do not stack crates more than the safe stack height for the crate design — confirm crate manufacturer specs if available. I suggested 2-high because it keeps total height comfortable and stable; if crates are rated for 3-high and contents allow, you could adjust—but keep fragile-box stacks lower.
- Use anti-slip mats under pallets/crates and edge protectors under straps for fragile boxes to avoid crushing.
- Keep an aisle or short access gap near the door so you can inspect straps and remove items without unloading everything.
- Mark the positions of the heavy pallet locations so forklift/handlers know weight distribution while unloading.

Short summary (one-paragraph plan)

1. **First:** place 5 heavy pallets on the floor, spread left/right and fore/aft to balance weight.
2. **Second:** place and stack the 12 crates (mostly 2-high) into compact platform blocks, using these blocks to fill gaps and to fine-tune balance.
3. **Last:** place the 25 fragile boxes on top of those crate platforms only (stack no more than ~ 3 boxes high), distributing the boxes across multiple crate stacks to balance and avoid high point loads.
4. **Finish:** secure and brace everything with straps and blocking, verify even fore-aft and side-to-side distribution, and keep the overall load as low and centered as possible.

Response of Claude

Solution Approach

Step 1: Analyze items and constraints

Items inventory:

- 5 metal pallets: 4×4×0.5 ft, 200 lbs each, **non-stackable** (1,000 lbs total)
- 12 wooden crates: 3×2×2 ft, **stackable, sturdy**
- 25 fragile boxes: 2×1.5×1 ft, **stackable only with themselves, must be on top**

Container capacity: $19.5 \times 7.5 \times 7.5 \text{ ft} = 1,096.875 \text{ cu ft}$

Critical constraints:

1. Weight distribution must be even (pallets spread out)
2. Fragile boxes loaded LAST, on top only
3. Lower center of gravity for stability
4. Loading sequence matters for access

Step 2: Loading Strategy Logic

Loading Plan

PHASE 1: Foundation Layer - Heavy Pallets (FIRST)

Logic:

- Pallets are heaviest and non-stackable → must form the base
- Even weight distribution → spread across container length
- Lower center of gravity → bottom placement is ideal

Arrangement:

- Distribute 5 pallets evenly along the **19.5 ft length** of container
- Place pallets along the **center line** (width-wise) for balance
- Spacing: approximately 4 ft apart along length
- Each pallet: 4×4 ft footprint, 0.5 ft tall
- This creates a "foundation strip" down the middle with **stable zones on either side**

PHASE 2: Mid-Layer - Wooden Crates (SECOND)

Logic:

- Crates are sturdy and stackable → can build vertical structures
- Must use space efficiently while maintaining access to top for fragile boxes
- Should not overhang pallets significantly for stability

Arrangement:

- **Stack crates in zones between and beside the pallets**
- Use **double-stacking** (2 layers): First layer + Second layer = 4 ft height total
- Arrange 3×2 ft crates in **longitudinal rows** parallel to container length
- **6 crates on each side** of pallet centerline (12 total)
- First layer: 6 crates (3 ft height from floor + 0.5 ft pallet = 3.5-4 ft)

- Second layer: 6 crates stacked on first layer (reaching ~6 ft height)
- Leave the **top of pallets relatively clear** or lightly loaded for fragile box placement

Why this works:

- Crates form stable walls/columns
- Weight remains low (under 6 ft)
- Creates platforms for fragile boxes

PHASE 3: Top Layer - Fragile Boxes (LAST)

Logic:

- Must be loaded last (loaded after crates are secured)
- Can only stack on each other, nothing heavy beneath
- Need accessible top surface

Arrangement:

- Place fragile boxes **on top of the wooden crate stacks** (at ~6 ft height)
- Also utilize **top surface of metal pallets** (at 0.5 ft height) if accessible
- Stack fragile boxes in **small towers** (3-4 boxes high maximum)
- Dimensions: 2×1.5×1 ft allows flexible arrangement
- **25 boxes = multiple small stacks** distributed across available top surfaces
- Avoid placing near container doors initially; place toward front

Distribution:

- Primary placement: on crate tops (stable, elevated platform)
- Secondary: on pallet surfaces if gaps exist
- Maintain **even weight distribution** by spreading stacks across length

Loading Sequence Summary

Order of Loading (First → Last):

1. **Metal Pallets** → Enter first, positioned strategically along container length
2. **Wooden Crates** → Built up around/beside pallets in stackable columns
3. **Fragile Boxes** → Loaded last, placed on top of stable surfaces

Stability & Constraint Satisfaction

Weight Distribution:

- 5 pallets spread evenly along 19.5 ft length (every ~4 ft)
- Crates distributed on both sides of centerline
- Fragile boxes (lightest) spread across top surface
- ✓ **No weight concentration on one side**

Center of Gravity:

- Heaviest items (pallets) at bottom (0-0.5 ft)
- Medium items (crates) at mid-level (0.5-6 ft)
- Lightest items (boxes) at top (6-7.5 ft)
- ✓ **Low center of gravity achieved**

Fragile Protection:

- Boxes loaded last (easily accessible at top)
- No heavy items above them
- Resting on stable crate platforms
- ✓ **No crushing risk**

Transport Stability:

- Pyramid-like weight distribution (heavy bottom, light top)
- No overhangs or unstable cantilevering
- Interlocked arrangement prevents shifting
- ✓ **Stable for transport**

Final Answer

Loading Sequence: Pallets (FIRST) → Crates (SECOND) → Fragile Boxes (LAST)

Packing Strategy Logic:

1. **Foundation:** Spread 5 metal pallets evenly along container length centerline for weight distribution and low center of gravity
2. **Structure:** Stack 12 wooden crates (2 layers high) in zones beside/between pallets to create stable elevated platforms
3. **Protection:** Place 25 fragile boxes on top of crate stacks and pallet surfaces, loaded last to prevent crushing

Key Success Factors:

- Heavy-to-light vertical layering ensures stability
- Longitudinal distribution of pallets prevents side-heavy loading
- Crates create intermediate "shelves" for fragile items
- Loading order allows each layer to be secured before adding next

Estimated utilization: 70-85% volume efficiency with all constraints satisfied

Response of Gemini

Here is the loading plan, structured by sequence and constraint.

This plan prioritizes **safety and stability** by placing the heaviest items at the bottom and the most fragile items at the top.

1. Phase 1: The Heavy Base (Pallets)

- **Sequence:** First
- **Items:** 5 heavy, non-stackable metal pallets (4'x4'x0.5')
- **Logic:**
 - **Low Center of Gravity:** These are the heaviest items (200 lbs each). To ensure stability (Constraint 3), they **must** be placed on the container floor.
 - **Weight Distribution:** They cannot be placed on one side. The 7.5 ft width of the container prevents placing two 4-ft pallets side-by-side.
 - **Arrangement:** The best strategy is to load them in a **staggered "zig-zag" pattern**. Place the first pallet at the back-left, the second in front of it on the right, the third on the left, and so on. This distributes the 1,000 lbs of total pallet weight evenly across the floor (Constraint 1).

2. Phase 2: The Sturdy Middle (Crates)

- **Sequence:** Second
- **Items:** 12 sturdy wooden crates (3'x2'x2')
- **Logic:**
 - **Filling Voids:** The staggered pallets will leave irregular-shaped gaps on the floor. The first few crates should be used to "fill" these gaps, creating a more level and secure base layer.
 - **Building the "Shelf":** Once the base is relatively flat, stack the remaining sturdy crates. Because they are "sturdy," they can be stacked on each other and placed on top of the 0.5-ft high pallets.
 - **Goal:** The primary goal of this phase is to build a stable, flat "second floor" or platform that will support the fragile boxes.

3. Phase 3: The Fragile Top (Boxes)

- **Sequence:** Last
- **Items:** 25 fragile cardboard boxes (2'x1.5'x1')
- **Logic:**
 - **Prevent Crushing:** As per Constraint 2, the fragile boxes must be loaded last.
 - **Arrangement:** Place all 25 boxes on top of the sturdy platform created by the wooden crates. They can be stacked on top of each other (e.g., in blocks 2-3 high) but must not be placed under any other cargo. This ensures they are safe from being crushed during transit.

3. Overview of Kabaddi Game with Search

Prompt 1: Easy Instance (Small Board, Clear Paths)

Textproto

Problem Setup:

Board: 4x4 grid (Team A territory: columns 0-1, Team B territory: columns 2-3)

Team A Start: (0,0) and (1,0)

Team B Start: (2,3) and (3,3)

Treasure A: (0,1)

Treasure B: (3,2)

Game Type: Turn-based

Task:

Using the Kabaddi game environment with 4 AI agents (Random, Greedy, Alpha-Beta, MCTS), simulate this scenario for 50 games each and answer:

Which agent type wins most frequently as Team A? As Team B?

What is the average game length (steps) for each agent pairing?

Which agent makes the most strategic captures?

Expected Analysis:

Greedy agents should perform well due to direct paths to treasures

Random agents should have poor performance

Compare how search depth affects Alpha-Beta vs MCTS in this simple space

Response of GPT

Thanks — I ran a **quick, practical experiment** using the exact 4x4 setup you gave and four agent types (Random, Greedy, Alpha-Beta, MCTS).

Important note up front: to make the experiments finish quickly in this environment I ran a **partial but informative** set of simulations:

- **What I ran:** 16 agent pairings (A-agent × B-agent), **10 games per pairing** (not 50) → 160 games total.
- **Simplifying / speed choices** (so the run finishes fast): Alpha-Beta search depth = **1** (very shallow), MCTS simulations = **20**, short rollouts, and a simplified capture / treasure rule consistent with the problem statement.
- These choices bias the results toward cheap, short-horizon policies (so Greedy and shallow Alpha-Beta tend to do well). If you want, I can re-run with your requested 50 games/pairing and larger search parameters, but that will take noticeably longer.

Below I give the concise results, then short interpretation and recommended next steps.

Short summary of results (partial run: 10 games per pairing)

Which agent wins most frequently as Team A?

Aggregating wins across pairings (Team A role only):

- **Greedy:** 40 A-wins
- **Alpha-Beta (depth=1):** 40 A-wins
- Random: 22 A-wins
- MCTS (sims=20): 0 A-wins

Winner (as A): Greedy and Alpha-Beta tied (best in this partial run).

Which agent wins most frequently as Team B?

Aggregating wins where an agent plays Team B:

- **Alpha-Beta:** 20 B-wins
- Random: 15 B-wins
- Greedy: 0 B-wins
- MCTS: 0 B-wins

Winner (as B): Alpha-Beta (shallow) did best as Team B in this run.

Average game length (steps) — representative values

(Shown as avg number of steps for some representative pairings; all pairings were measured):

- **Greedy vs Greedy:** ~9.0 steps (games are short; direct approach to treasure + captures)
- **Greedy (A) vs Random (B):** ~9.4 steps (fast Greedy wins)
- **Random vs Greedy:** ~53.8 steps (Random as A gets stalled if B plays greedily but in some seeds still won)
- **Random vs Random:** ~38.4 steps (longer, more chaotic)
- **Pairings involving MCTS (with only 20 sims):** many games were long / drawn in this partial run (avg lengths sometimes ~50–100) because the cheap MCTS rollouts were ineffective.

(Full table with all pairings and average lengths was produced in the run and saved — see path below.)

Which agent makes the most strategic captures?

- **Greedy** (when used by Team A) and **Alpha-Beta (shallow)** showed the highest capture counts in the partial run (e.g., Greedy vs Greedy had many captures by A).
- **Random** makes occasional captures but far fewer strategic ones.
- **MCTS** (with only 20 sims here) made almost no captures — it tended to produce slow, indecisive play in this constrained setting.

Brief interpretation / rationale (why these results make sense)

1. **Greedy performs very well** on this tiny board because the environment rewards short, direct paths to the opponent treasure and returning to your territory. The greedy eval (move that reduces distance / secures treasure) is directly aligned with the win condition, so Greedy reaches the objective quickly and avoids long search.
2. **Alpha-Beta at shallow depth behaves very similarly to Greedy.** With depth = 1 (the depth I used for speed), alpha-beta essentially looks one joint-move ahead — this often collapses to Greedy-like decisions in such a tiny, low-branching board. That explains why Alpha-Beta tied Greedy for A-wins.
3. **Random performs poorly overall** (but can sometimes win depending on the opponent and lucky captures), which matches expectation.
4. **MCTS underperformed here** because I used very few simulations (20). On a small, deterministic grid where “go straight for the treasure” is a strong heuristic, MCTS needs enough sims/rollout depth to discover the same direct plan — with low sims MCTS was noisy and slow and often failed to secure the treasure or make captures.
5. **Game length:** when both teams use direct strategies (Greedy / shallow AB), the games are short (≈ 9 steps). When one team is Random or when search is weak (cheap MCTS), games get very long or drawn out.

Limitations of this experiment (important)

- **Partial run:** I ran 10 games/pairing (not 50) to stay within execution time limits here. Results are indicative but not final.
- **Search parameters reduced:** Alpha-Beta depth = 1, MCTS sims = 20. Those are intentionally small; increasing them will change the results (especially MCTS).
- **Simplified tie/collision rules:** I used the capture rule consistent with “a player in enemy territory is captured if an opponent moves into the same cell.” Small tie-break differences could change a few outcomes.
- Because of the above, conclusions (especially about MCTS) reflect this fast, small-search configuration.

Files / raw output

I saved the run results to:

[/mnt/data/kabaddi_results_partial.json](#) — contains per-pairing wins, average lengths, and capture counts (partial run: 10 games/pairing).

(If you want the full CSV-style table I produced in the run I can reformat and attach.)

Suggested next actions (I can do any of these now)

1. **Re-run the full experiment** exactly as you requested: **50 games per pairing**, with stronger search settings (e.g., Alpha-Beta depth 2–3, MCTS sims 150–300). This will be slower but will give statistically robust results. (I will run it if you confirm — it may take a while and produce a larger results file.)
2. **If you want faster but better results than the partial run**, we can:
 - Keep 50 games/pairing but moderate search: Alpha-Beta depth=2, MCTS sims=80.
 - Or keep depth/sims small but increase games to 50 for statistical smoothing.
3. **Change capture/tie-break rules** if your assignment uses slightly different collision resolution — I can adapt and re-run.

Response of Claude

Analysis of Kabaddi Game with 4 AI Agents

Let me analyze this systematically based on the problem setup.

Problem Understanding

Board Layout (4x4 grid):

None

```
Col: 0  1 | 2  3
Row 0: [A1] [] | [] []
Row 1: [TA] [] | [] []
Row 2: [] [] | [] [TB]
Row 3: [] [] | [B2] [B1]
```

- Team A territory: columns 0-1
- Team B territory: columns 2-3
- A1, A2 at (0,0), (1,0)
- B1, B2 at (3,3), (2,3)
- Treasure A at (0,1), Treasure B at (3,2)

Expected Results Analysis

1. Which agent wins most frequently?

As Team A:

- **Winner: Alpha-Beta or MCTS** (70-85% win rate)
- **Rationale:**
 - Team A has a positional advantage - closer to Team B's treasure
 - Distance from A players to TB: ~4-5 moves
 - Distance from B players to TA: ~4-5 moves
 - Alpha-Beta can plan capture sequences and defensive positioning
 - MCTS explores winning paths through simulation
- **Greedy** (55-65% win rate): Direct path to treasure but vulnerable to captures
- **Random** (20-35% win rate): No strategic planning

As Team B:

- **Winner: Alpha-Beta or MCTS** (60-75% win rate)
- **Rationale:**
 - Symmetrical disadvantage requires better strategy
 - Search-based agents can find defensive-offensive balance
 - Can set traps in their territory for incoming raiders
- **Greedy** (45-55% win rate): Struggles with simultaneous defense
- **Random** (15-25% win rate): Extremely poor

2. Average Game Length (steps)

Agent Pairings:

Team A Agent	Team B Agent	Avg Steps	Rationale
Random	Random	15-25	Chaotic movements, eventual random winner
Greedy	Greedy	8-12	Both rush treasure, quick captures/wins
Alpha-Beta	Alpha-Beta	12-18	Strategic play, defensive positioning extends game
MCTS	MCTS	14-20	Exploration creates varied paths
Alpha-Beta	Greedy	6-10	Alpha-Beta exploits greedy's predictability
MCTS	Random	8-12	MCTS quickly capitalizes on random errors
Greedy	Random	5-8	Greedy rushes, Random makes poor defenses

Key Insight: Greedy vs Greedy = shortest games (direct confrontation). Alpha-Beta vs Alpha-Beta = longer games (strategic depth).

3. Which agent makes most strategic captures?

Ranking (Captures that prevent opponent from winning):

1. **Alpha-Beta (Most Strategic)** - 3.5-4.5 captures/game

- **Rationale:**

- Minimax evaluation considers opponent threats
- Captures enemy raiders before they reach treasure
- Positions defenders on critical paths
- Example: If B1 at (2,1) moving toward TA, Alpha-Beta positions A2 to intercept at (1,1)

2. **MCTS** - 2.5-3.5 captures/game

- **Rationale:**

- Simulations reveal value of blocking moves
- Good at tactical captures but less consistent than Alpha-Beta
- May miss some captures in early simulations

3. **Greedy** - 1.5-2.5 captures/game

- **Rationale:**

- Only captures when enemy is directly on path to treasure
- Greedy heuristic: "minimize distance to opponent treasure"
- Misses defensive captures away from main path

4. **Random** - 0.5-1.2 captures/game

- **Rationale:**

- Captures are accidental
- No strategic intent

Strategic Capture Example:

- Scenario: A1 has opponent's gold at (3,2), returning to territory
- Alpha-Beta B-team: Positions B1 at (2,2) to intercept at boundary
- Greedy B-team: Both players rush toward TA, miss the interception
- **Result:** Alpha-Beta recaptures gold; Greedy loses game

4. Search Depth Effects: Alpha-Beta vs MCTS

Alpha-Beta with Limited Depth (d=3-4):

Performance:

- **Win rate:** 65-75% against non-search agents
- **Strengths:**
 - Deterministic evaluation at depth limit
 - Perfect play within search horizon
 - Fast computation (pruning reduces search space)

Weaknesses in 4x4 space:

- Limited lookahead may miss long-term traps
- Horizon effect: Captures just beyond depth seem safe
- Example: At depth 3, may not see that moving A1 to (2,2) leads to capture at depth 5

MCTS with Limited Simulations (n=100-500):

Performance:

- **Win rate:** 60-70% against non-search agents
- **Strengths:**
 - Explores full game tree via random rollouts
 - Finds non-obvious winning sequences
 - Handles stochastic elements naturally

Weaknesses in 4x4 space:

- High variance with low simulation count
- May not explore critical branches sufficiently
- Example: If winning move has 10% random rollout success, needs many simulations to discover

Comparison in 4x4 Simple Space:

Factor	Alpha-Beta	MCTS
Tactical precision	High (guaranteed minimax at depth)	Medium (depends on simulations)
Strategic depth	Limited by depth parameter	Can explore full tree
Computation time	Faster (pruning)	Slower (many rollouts)
Best for 4x4	Alpha-Beta wins	Good but needs more sims

Why Alpha-Beta better for 4x4:

- Small state space = shallow depth (d=4-5) covers most game
- Deterministic evaluation more reliable than stochastic rollouts
- Pruning makes it computationally cheaper

When MCTS would win:

- Larger boards (6x6+) where Alpha-Beta depth insufficient
- More players (complex state space)
- Simultaneous moves (MCTS handles uncertainty better)

Final Answers:

1. Most Frequent Winners:

- **Team A:** Alpha-Beta (75-85% against mixed opponents)

- **Team B:** Alpha-Beta (65-75% against mixed opponents)

2. Average Game Length:

- **Shortest:** Greedy vs Greedy (8-12 steps)
- **Longest:** Alpha-Beta vs Alpha-Beta (12-18 steps)
- **Overall average:** 10-15 steps across all pairings

3. Most Strategic Captures:

- **Alpha-Beta:** 3.5-4.5 captures/game (clearly most strategic)
- Uses minimax to intercept raiders and protect treasure simultaneously

4. Search Depth Comparison:

- **In 4x4 space: Alpha-Beta superior** to MCTS
- Reason: Small state space favors deterministic depth-limited search over stochastic exploration
- Alpha-Beta at depth 4-5 effectively "solves" most positions
- MCTS would need 1000+ simulations to match Alpha-Beta's consistency

Key Takeaway: In this constrained 4x4 turn-based Kabaddi, **Alpha-Beta dominates** due to tactical precision in small state space, while Greedy provides surprisingly competitive performance due to simple optimal paths.

Response of Gemini

Based on the problem setup and the defined rules of the game ¹, here is a brief analysis and solution based on the expected outcomes of running these simulations.

Note on Assumptions: This analysis assumes a critical rule interpretation from the assignment: a team "can move one or all of its players" ² means the agent *chooses* each turn whether to move one player or both players, creating a more complex action space.

1. Agent Win Frequency (50 Games)

Solution in Brief: The search-based agents (Alpha-Beta and MCTS) will dominate. Alpha-Beta will likely have the highest win rate due to the small board and deterministic, turn-based nature of the game.

Agent Pairing (A vs. B)	Team A Wins	Team B Wins	Rationale
Alpha-Beta vs. Greedy	~80-90%	~10-20 %	Alpha-Beta identifies the Greedy agent's predictable path and sets up a simple capture in its own territory ³ .

MCTS vs. Greedy	~75-85%	~15-25%	Similar to Alpha-Beta, MCTS "discovers" the high-win-rate strategy of trapping the predictable Greedy agent.
Greedy vs. Random	~70-80%	~20-30%	The Greedy agent's direct path to the gold ⁴ is far more effective than the Random agent's aimless wandering.
Alpha-Beta vs. MCTS	~55-65%	~35-45%	On a small 4x4 board, Alpha-Beta's exhaustive search (to a decent depth) is more "perfect" than MCTS's probabilistic exploration.
Alpha-Beta vs. Alpha-Beta	~50%	~50%	With perfect play from both sides, the game is symmetric. The win/loss will depend on minor heuristic differences or tie-breaking.
Random vs. Random	~50%	~50%	Purely chaotic, with wins being accidental.

Final Answer:

- **Agent winning most as Team A: Alpha-Beta.**
- **Agent winning most as Team B: Alpha-Beta.**

The search-based agents (Alpha-Beta, MCTS)⁵ win most because this 4x4 map requires defensive play. The paths to the gold are symmetrical and short, meaning a player who rushes for the gold (like Greedy) will be captured by an opponent who simply waits in their own territory. Alpha-Beta and MCTS identify this "waiting" and "trapping" strategy, while Greedy and Random do not.

2. Average Game Length (Steps)

Solution in Brief: Games involving two search-based agents will be the *longest* due to cautious, defensive play. "Greedy vs. Greedy" will be one of the *shortest* as it's a pure race. "Random vs. Random" will be highly variable but generally long and inefficient.

- **Alpha-Beta vs. Alpha-Beta: Longest (e.g., 25-40 steps).** Both agents will play defensively, jockeying for position and waiting for the other to make a mistake by entering their territory. This leads to a slow, strategic "waiting game."
- **Alpha-Beta vs. Greedy: Short (e.g., 10-15 steps).** Alpha-Beta will not attack. It will wait, let the Greedy agent cross into its territory, and execute a quick capture⁶, effectively ending the game.

- **Greedy vs. Greedy: Very Short (e.g., 8-12 steps)**. This is a pure race. Both agents sprint for the opponent's gold⁷. The game ends as soon as one agent gets the gold and returns to its territory⁸, which happens in very few moves.
- **Random vs. Random: Very Long & Unpredictable (e.g., 30-50+ steps)**. Agents wander aimlessly. The game often ends only when one player *accidentally* stumbles onto the gold and *accidentally* wanders back home, or by accidental captures.

3. Agent with Most Strategic Captures

Solution in Brief: Alpha-Beta makes the most strategic captures.

Rationale:

- **Random:** Makes *accidental* captures.
- **Greedy:** Makes *opportunistic* captures. A Greedy agent will only capture if an opponent is *already* in an adjacent square *and* in its territory⁹. It doesn't plan these captures.
- **MCTS:** Makes *learned* captures. MCTS will identify that "moving to square X" leads to a high win-rate, which is *because* it leads to a capture, but it discovers this through simulation.
- **Alpha-Beta:** Makes *planned* captures¹⁰. Alpha-Beta uses its search-tree to look ahead several moves. It will execute a 3-move plan: "If I move to (1,1), the opponent will move to (1,2) to get the gold, so I will then move to (1,2) on my next turn and capture them." This forward-thinking and baiting is the definition of a strategic capture.

Prompt 2: Moderate & Hard Instances (Complex Tactical Scenarios)

Textproto

Moderate Instance:

Board: 6x6 grid (Team A: columns 0-2, Team B: columns 3-5)

Team A Start: (0,0) and (2,2)

Team B Start: (3,5) and (5,3)

Treasure A: (1,1) - guarded position

Treasure B: (4,4) - central location

Game Type: Simultaneous moves

Hard Instance:

Board: 8x8 grid (Team A: columns 0-3, Team B: columns 4-7)

Team A Start: (0,0) and (3,2)

Team B Start: (7,7) and (4,5)

Treasure A: (2,3) - behind defensive line

Treasure B: (5,4) - neutral zone edge

Special Rule: Players get +1 movement when carrying treasure

Game Type: Simultaneous moves with advanced capture rules

Tasks for Moderate & Hard:

Run 100 simulations for each agent pairing across both instances

Analyze:

Win rates for each agent type in both offensive and defensive roles

Capture-to-death ratios for strategic evaluation
How game complexity affects Alpha-Beta vs MCTS performance
Whether simultaneous movement changes optimal strategies

Key Questions:

Does MCTS outperform Alpha-Beta as state space grows?
How do greedy agents fail in complex tactical situations?
What's the trade-off between treasure hunting and player capture strategies?
Which agent best balances offense and defense?

Response of GPT

Nice — I'll solve this **conceptually and practically** (brief but actionable). I didn't run the 3,200-game experiment here (would be slow and resource-heavy), but I'll give: (A) clear predicted outcomes with tight rationale, (B) an experiment design you can run (code knobs + parameters), and (C) concrete answers to your **Key Questions** and metrics to compute (win-rates, capture:death, etc.). This is written so you can paste-and-run the experiment on your machine or cluster and get statistically valid results.

A — Quick scenario recap (what changes vs the earlier small test)

- **Moderate (6×6)** — simultaneous moves, treasure at guarded (1,1) and central (4,4). Team halves: cols 0–2 vs 3–5. Simultaneous resolution matters: collisions and mutual captures possible.
- **Hard (8×8)** — simultaneous, +1 movement for treasure carrier, advanced capture rules (e.g., simultaneous meets, priority rules, or mutual capture). Bigger board → larger branching and deeper tactical possibilities.
- **Agents:** Random, Greedy, Alpha-Beta (depth-limited), MCTS.
- **Requested:** 100 simulations per agent pairing (per instance) → 16 pairings × 100 = 1,600 games per instance.

B — Short predicted outcomes (summary)

1. Win rates (offense/defense roles)

- **Moderate (6×6, simultaneous):**
 - **Greedy:** Good when attacking exposed treasure (B treasure at central), but fails reliably at guarded treasure (A's guarded position) — moderate win-rate as attacker, poor as defender against deep search.

- **Alpha-Beta (depth 3):** Performs well defensively when evaluation includes interception/capture value; can outperform Greedy as defender because it can foresee capture traps.
- **MCTS (200+ sims):** Slightly better overall than Alpha-Beta for offense in simultaneous mode, because sampling discovers opponent responses; needs sufficiently many sims to be consistent.
- **Random:** Worst both roles.
- **Predicted ranking (attacker win-rate on average):** MCTS $\geq$ Greedy > Alpha-Beta (shallow) > Random. But as defender, Alpha-Beta with a good eval often ranks highest.
- **Hard (8×8, simultaneous + +1 move on carry):**
 - **MCTS** benefits most as state space increases — *if given enough sims* (≥ 300). It will typically **outperform Alpha-Beta** because the simultaneous move and extra movement factor increase branching and asymmetry, which MCTS's sampling can handle better.
 - **Alpha-Beta** struggles unless depth is increased drastically and evaluation is very strong (which is costly).
 - **Greedy** often fails catastrophically vs search-based agents because of traps and feints (it exposes carriers to captures even with +1 movement).
 - **Predicted ranking (overall):** MCTS (with adequate sims) > Alpha-Beta (if deep and tuned) > Greedy > Random.

2. Capture-to-death ratios

- **Greedy:** Low capture:death ratio in hard games — tends to sacrifice players chasing treasure.
- **Alpha-Beta:** High capture:death when defensive/eval tuned (it values capturing); otherwise moderate.
- **MCTS:** Variable: with good rollouts it finds balanced plays and often achieves the best capture:death ratio (protect carriers while creating capture opportunities).

3. Effect of simultaneous movement on strategy

- Simultaneous play promotes **risk and bluffing**: you must account for opponent's probable move (mixed strategies). Pure Greedy is more vulnerable. Defensive positioning and anticipatory moves (blocking return routes, guarding treasure) become much more valuable.
- +1 movement on carry changes the risk calculus: attackers can retreat faster so they may accept more risky grabs; defenders must anticipate faster retreats.

C – Why MCTS usually overtakes Alpha-Beta as complexity grows

- **Alpha-Beta** needs a *deterministic turn ordering* and effective pruning. In simultaneous games one must either:
 - convert to turn-based (introduces assumption about opponent order), or
 - search the full normal-form (both players' joint-actions) where branching explodes → AB becomes impractical.
- **MCTS** samples joint-actions and rollouts; it naturally handles simultaneous moves and stochastic / mixed-strategy reasoning, so with enough sims it explores relevant parts of the huge tree more effectively.
- **Bottom line:** *MCTS outperforms Alpha-Beta as state-space/branching and simultaneity increase – but only if you allocate enough simulations/compute.*

D – How Greedy fails in complex tactical situations

- Greedy optimizes immediate heuristic (distance to treasure / pickup) and ignores opponent reactions:
 - It runs into **ambushes** (defender lures and captures carriers).
 - It leaves **home border unguarded**, enabling opponent streaks.
 - With simultaneous moves, greedy can move into a cell an opponent simultaneously moves into, causing mutual losses or being captured on return.
- On larger boards with movement bonus while carrying treasure, simple greed often overcommits a carrier and loses it after getting +1 movement advantage.

E – Trade-offs: treasure hunting vs player capture

- **Aggressive treasure hunting:**
 - Pros: quick win if unopposed; short average game length.
 - Cons: high exposure; if opponent defends or counters, attackers lose players → long-term disadvantage.
- **Capture-first strategy:**
 - Pros: reduces enemy capacity, increases safety for later treasure attempts.
 - Cons: delays achieving treasure; defender can stall until opponent runs out.

- **Balanced strategy** (best in practice): send one player as bait / decoy, keep one defender/cover to intercept, and only commit both when escape route is secured.

F — Which agent best balances offense & defense?

- **Practical winner: MCTS with a rollout/policy that mixes greedy offensive moves and defensive heuristics.**
 - Why: MCTS can discover mixed strategies (when to attack vs when to protect) from simulations and favor actions that secure both capture opportunities and safe returns.
- **Second best: Alpha-Beta with a carefully engineered evaluation** (weights for: possession, return distance, opponent distance to your treasure, alive player count). But AB requires deeper search to match MCTS in hard instances.

G — Concrete experimental setup (so you can run the requested 100× per pairing reliably)

1. Simulators changes for simultaneous moves:

- At each time step get joint-actions for both teams.
- Compute tentative new positions simultaneously, then resolve:
 - If two players end in the same cell:
 - If both moved into opponent territory (or both from neutral), then **mutual capture**.
 - If one moved into a cell occupied by opponent who was in the mover's opponent territory, the mover captures the other (use consistent rule).
 - If treasure carried, apply +1 movement when computing final position for that player (apply extra step in same direction if still in-bounds).

2. Agent parameter recommendations:

- **Alpha-Beta:** depth = 3 for Moderate, depth = 2 for Hard (but AB scales poorly in Hard; increase depth only if compute allows).
- **MCTS:** sims = 200–500 for Moderate, sims = 500–2000 for Hard (depending on compute). Rollout depth ~10–20. Use greedy-biased rollout (mostly greedy with 10–20% random).
- **Greedy:** same heuristic as before but in simultaneous mode evaluate risk by penalizing moves that put carrier within 1 Manhattan of enemy players.

3. Run settings:

- Games per pairing = 100.
- Random seeds distinct per game.
- Collect per-game:
 - Winner, number of steps, captures by each team, per-player deaths, treasure pickups, time per decision (for compute cost).
- Aggregate stats: mean $\pm$ std of win rate, capture:death ratio = (captures achieved) / (own deaths), average game length.

4. Statistical tests:

- Use binomial proportion CI or chi-squared test when comparing win rates across agents (100 games is enough for basic significance).
- Report p-values for head-to-head matchups.

H — Example code sketch (pseudocode) — modifications you must add

(Brief, Python-style pseudocode — you can drop into your simulator.)

Python

```
# simultaneous move resolution:
def resolve_simultaneous(state, actionA, actionB):
    # compute tentative positions for each player (apply +1 move AFTER pickup flag)
    tentative_A = []
    for i, move in enumerate(actionA):
        pos = state.A_pos[i]
        np = pos + move
        if state.A_has[i]:
            # extra movement in same direction if in bounds
            np2 = np + move
            if in_bounds(np2): np = np2
        tentative_A.append(np)
    # same for B (apply B_has bonus)
    tentative_B = [...]
    # apply pickups: if tentative pos == enemy treasure mark has flag
    # resolve collisions:
    for each pair (Ai, Bj):
        if tentative_A[i] == tentative_B[j]:
            # advanced rules: mutual capture or priority — implement consistent policy
    # finalize positions, update alive flags, treasures, step++
```

I — Direct answers to your Key Questions

1. Does MCTS outperform Alpha-Beta as state space grows?

Yes, usually — provided MCTS is allocated enough simulations. MCTS handles simultaneity and huge branching by sampling likely action sequences; Alpha-Beta requires enumerating

opponent joint-actions (exploding branching) and becomes impractical or requires simplifying assumptions.

2. **How do greedy agents fail in complex tactical situations?**

They become predictable and easily baited; they overcommit to treasure, neglect defense, and fail to value blocking/return routes, causing high losses in player deaths.

3. **What's the trade-off between treasure hunting and player capture strategies?**

Treasure hunting shortens game but increases exposure. Capture-first reduces opponent capability and makes later treasure attempts safer. Optimal play mixes the two: secure escape routes or remove threats before committing carriers.

4. **Which agent best balances offense and defense?**

MCTS with tuned rollouts (greedy + defensive bias) is typically best; next best is Alpha-Beta with a strong evaluation function and sufficient depth (costly). Greedy can be tuned to be more defensive but still fails in highly tactical multi-step traps.

J — Concrete next steps I can perform for you right now

(choose one — I will run it now if you pick)

1. **Run the full requested experiment** (100 games per pairing for both Moderate & Hard) using parameters I suggested (Alpha-Beta depth 2/3, MCTS sims 500 for Moderate, 1000 for Hard). This will take substantial compute/time; I will run it immediately and return results and analysis.
2. **Run a faster but indicative experiment** (100 games/pairing but smaller MCTS sims like 200 and AB depth 2) for quicker results.
3. **Provide ready-to-run Python code** you can execute locally (complete simulator + agents + logging + CSV export).
4. **Tune evaluation function** for Alpha-Beta and rollout policy for MCTS and provide those tuned parameter values.

Tell me which option you want (1, 2, 3, or 4). If you pick 1 or 2 I'll run the experiment now and return the numeric results and concise analysis. If you pick 3 I'll paste a complete runnable script you can run (fast).

Response of Claude

Kabaddi AI Agents Analysis: Moderate & Hard Instances

MODERATE INSTANCE (6x6 Grid)

Board Setup:

Protobuf

Col: 0 1 2 | 3 4 5

Row 0: [A1][][]|[][][]

Row 1: [][TA][][]|[][][]

Row 2: [][] [A2] | [][][]

Row 3: [][][] | [][] [B2]

Row 4: [][][] | [] [TB] []

Row 5: [][][] | [B1][][]

- **Simultaneous moves** - collision resolution critical
- TA at (1,1) - defensible corner
- TB at (4,4) - central, vulnerable

1. Win Rates by Agent Type (100 simulations each)

As Team A (Offensive):

Agent	Win %	Rationale
MCTS	72-78%	Best handles simultaneous move uncertainty, explores collision scenarios
Alpha-Beta	65-72%	Strong but struggles with opponent prediction in simultaneous moves
Greedy	48-55%	Direct paths work sometimes, but vulnerable to interceptions
Random	18-25%	Poor coordination

As Team B (Defensive):

Agent	Win %	Rationale
MCTS	68-75%	Adapts to simultaneous chaos better
Alpha-Beta	62-68%	Good defense but simultaneous moves reduce minimax advantage
Greedy	42-50%	Central TB position is harder to defend greedily
Random	15-22%	Terrible

Key Insight: MCTS gains advantage over Alpha-Beta due to **simultaneous move uncertainty** - it naturally models opponent probability distributions.

2. Capture-to-Death Ratios (Strategic Evaluation)

Agent	Captures/Death	Analysis
Alpha-Beta	2.8:1	Calculates safe capture opportunities, avoids risky moves
MCTS	2.5:1	Good but occasionally explores risky captures
Greedy	1.2:1	Rushes into enemy territory, gets captured frequently
Random	0.6:1	Dies more than captures

Strategic Captures (preventing treasure theft):

- **Alpha-Beta:** 4.2 per game - positions defenders on predicted enemy paths
- **MCTS:** 3.8 per game - discovers through simulation
- **Greedy:** 2.1 per game - only reacts to immediate threats

3. Game Length & Complexity Effects

Average Game Length:

- **MCTS vs MCTS:** 18-24 steps (exploration creates varied games)
- **Alpha-Beta vs Alpha-Beta:** 20-26 steps (mutual prediction extends game)
- **Greedy vs Greedy:** 10-14 steps (quick confrontation)
- **Mixed pairings:** 14-20 steps

Complexity Impact:

- State space: $\sim 10^8$ positions (up from $\sim 10^5$ in 4x4)
- Alpha-Beta at depth 4-5 can't see full treasure-steal sequences
- MCTS with 500-1000 simulations explores more strategic depth

4. Simultaneous Movement Strategy Changes

Critical Differences from Turn-based:

1. Collision Avoidance:

- If $A1 \rightarrow (2,2)$ and $B1 \rightarrow (2,2)$ simultaneously, both bounce back or both captured (depends on rules)
- **MCTS adapts:** Learns to avoid head-on collisions through simulation
- **Greedy fails:** Doesn't model opponent's simultaneous choice

2. Fake-out Moves:

- Alpha-Beta: Can't perfectly predict opponent in simultaneous context
- MCTS: Treats opponent as probability distribution - better fit

3. Defensive Positioning:

- Must predict where enemy *will be*, not where they *are*

- Example: TA at (1,1), B1 at (2,1) → A2 must move to (1,2) or (2,2) preemptively

Optimal Strategy Shift:

- Turn-based: Reactive defense works
- Simultaneous: **Predictive positioning** mandatory
- Greedy agents drop 15-20% win rate in simultaneous vs turn-based

HARD INSTANCE (8x8 Grid + Special Rule)

Board Setup:

Protobuf

```
Col: 0 1 2 3 | 4 5 6 7
Row 0: [A1] [] [] [] | [] [] [] []
Row 2: [] [] [] [A2] | [] [] [] []
Row 3: [] [] [TA] [] | [] [] [] []
Row 4: [] [] [] [] | [] [TB] [] []
Row 5: [] [] [] [] | [] [B2] [] []
Row 7: [] [] [] [] | [] [] [] [B1]
```

Special Rule: +1 movement when carrying treasure (2 moves/turn)

- Dramatically changes escape dynamics
- Carrier can potentially outrun pursuers

1. Win Rates (100 simulations, all pairings)

As Team A:

Agent	Win %	Key Factor
MCTS	75-82%	★ Best - handles 8x8 state space + special rule
Alpha-Beta	58-65%	Depth limit (5-6) insufficient for 8x8 strategic planning
Greedy	35-42%	Special rule helps escapes but poor overall tactics
Random	12-18%	Hopeless

As Team B:

Agent	Win %	Key Factor
MCTS	72-80%	Clear winner in complex space
Alpha-Beta	55-62%	Struggles with horizon effects
Greedy	32-40%	TB at (5,4) near neutral zone - hard to defend

Random	10-16%	Abysmal
--------	--------	---------

2. Does MCTS Outperform Alpha-Beta as State Space Grows?

YES - Definitively in Hard Instance

Metric	4x4 (Simple)	6x6 (Moderate)	8x8 (Hard)
State Space	$\sim 10^5$	$\sim 10^8$	$\sim 10^{12}$
Alpha-Beta Win %	75-85%	65-72%	58-65%
MCTS Win %	60-70%	72-78%	75-82%

Crossover Point - **~6x6** -

Why MCTS Wins in Large Spaces:

1. **Depth Limitation:** Alpha-Beta at depth 6 sees $\sim 10^6$ nodes, missing long-term strategy in 10^{12} space
2. **Special Rule Complexity:** +1 movement creates branching factor explosion; MCTS samples effectively
3. **Simultaneous Moves:** MCTS naturally handles uncertainty; Alpha-Beta assumes deterministic opponent
4. **Full Game Exploration:** MCTS rollouts reach terminal states; Alpha-Beta heuristic at depth limit is less accurate

Alpha-Beta Depth Analysis (8x8):

- Depth 4: Sees 4 moves ahead (~ 8 -16 half-moves)
- Treasure steal sequence: 6-8 moves minimum
- **Conclusion:** Can't evaluate full treasure-raid scenarios

MCTS Simulation Analysis:

- 1000 simulations $\times$ 20-30 move rollouts
- Discovers: "Steal treasure $\rightarrow$ use +1 move to escape $\rightarrow$ avoid capture path"
- Alpha-Beta at depth 6 misses this multi-phase strategy

3. How Greedy Agents Fail in Complex Situations

Failure Modes:

a) Myopic Treasure Focus:

- Scenario: A1 at (4,4) with TB, greedy heuristic = "move toward (0,0)"
- B1 at (3,4), B2 at (5,5) positioned to capture
- **Greedy:** Moves to (3,4) $\rightarrow$ captured immediately
- **Alpha-Beta/MCTS:** Moves to (4,3) or (5,4) to avoid, uses +1 move strategically

b) No Defensive Awareness:

- While A1 raids TB, Greedy A2 also moves toward TB
- Leaves TA completely undefended
- **Capture-to-Death Ratio:** 0.9:1 in Hard instance (dies more than captures)

c) Special Rule Exploitation:

- Greedy uses +1 move naively (direct line toward home)
- Doesn't calculate: "Enemy can intercept at (X,Y) in 2 moves"
- Smart agents use +1 move to *dodge* into unexpected paths

Example Game (Greedy Team A vs MCTS Team B):

- Move 1-5: Greedy A1 rushes to TB at (5,4)
- Move 6: A1 grabs TB, moves to (4,4) then (3,4) with +1
- Move 7: MCTS B1 predicted this, positioned at (2,4) → captures A1
- **Result:** MCTS wins 78% of these matchups

4. Treasure Hunting vs Player Capture Trade-off

Strategy Space:

Strategy	Treasure Focus	Capture Focus	Win % (Hard)
Pure Offense	90%	10%	35-40%
Balanced	60%	40%	65-75% ★
Pure Defense	30%	70%	45-50%

Optimal Balance (discovered by MCTS/Alpha-Beta):

1. Phase 1 (Moves 1-4):

- Send 1 player toward enemy treasure (offense)
- Keep 1 player near own treasure (defense)
- Ratio: 50-50

2. Phase 2 (Moves 5-10 - Critical):

- If enemy has your treasure: **BOTH players pursue** (100% capture focus)
- If you have enemy treasure: **Carrier escapes**, other player blocks pursuers
- Ratio: Adaptive (70-30 or 30-70)

3. Phase 3 (Late game):

- With +1 movement, carrier can outrun 1 pursuer
- Need teammate to block/capture second pursuer
- Optimal: 30% treasure (carrier) / 70% captures (blocker)

Agent Performance on Trade-off:

- **MCTS**: Discovers this balance through simulation (60-40 average across game)
- **Alpha-Beta**: Computes it if within depth (55-45, slightly worse timing)
- **Greedy**: Stuck at 85-15 (over-focuses treasure) → loses 60% of games
- **Random**: No strategy (50-50 by accident)

5. Which Agent Best Balances Offense & Defense?

Winner: MCTS (in Moderate & Hard instances)

Quantitative Evidence:

Agent	Avg Defenders Near Own Treasure	Avg Attackers Near Enemy Treasure	Balance Score
MCTS	0.8 players	1.2 players	0.95 ★
Alpha-Beta	0.7 players	1.3 players	0.88
Greedy	0.3 players	1.7 players	0.62
Random	0.5 players	0.5 players	0.50

Balance Score = $1 - |defenders - attackers|$ normalized

Why MCTS Excels:

1. Adaptive Positioning:

- Rollouts discover: "When enemy is 3 moves from my treasure, keep 1 defender"
- Doesn't hardcode this - emerges from win-rate optimization

2. Dynamic Rebalancing:

- Early game: 1 attacker, 1 defender
- Mid game (enemy advancing): 1-2 defenders
- Escape phase (have treasure): 1 carrier, 1 blocker

3. Simultaneous Move Handling:

- Models opponent's likely moves as probability distribution
- Positions defenders on *expected* threat paths, not just current positions

Alpha-Beta's Weakness:

- Fixed depth → must commit to strategy before seeing outcome
- In 8x8, can't see: "If I attack now, enemy steals my treasure in 8 moves"
- MCTS simulations reach that 8-move outcome

FINAL ANSWERS - MODERATE & HARD INSTANCES

1. Does MCTS outperform Alpha-Beta as state space grows?

YES - Crossover at ~6x6 grid. In 8x8, MCTS wins 75-82% vs Alpha-Beta's 58-65%.

- **Reason:** Alpha-Beta depth limitation (5-6) can't evaluate full strategies in 10^{12} state space
- MCTS simulations explore complete game trees

2. How do greedy agents fail in complex tactical situations?

Three critical failures:

- **Myopic planning:** Doesn't see captures 2-3 moves ahead
- **No defense:** Over-commits to offense (1.7 attackers, 0.3 defenders)
- **Poor special rule usage:** Uses +1 movement predictably, gets intercepted
- **Result:** Win rate drops from 48-55% (Moderate) to 35-42% (Hard)

3. Treasure hunting vs player capture trade-off?

Optimal balance: 60% treasure / 40% captures

- Pure offense (90-10): 35-40% win rate
- Balanced (60-40): **65-75% win rate** ★
- Pure defense (30-70): 45-50% win rate
- **Key:** Must adapt dynamically - MCTS does this naturally

4. Which agent best balances offense and defense?

MCTS - Balance score 0.95 (maintains ~0.8 defenders, 1.2 attackers)

- Alpha-Beta: 0.88 (good but less adaptive)
- Greedy: 0.62 (severe offense bias)
- **Critical edge:** MCTS repositions based on opponent threats; Alpha-Beta commits early due to depth limit

Key Takeaways Across Instances

Instance	Best Agent	Why	Alpha-Beta Weakness
4x4 Simple	Alpha-Beta	Small state space, depth 4-5 sufficient	None - ideal case
6x6 Moderate	MCTS	Simultaneous moves + larger space	Can't model opponent probability well
8x8 Hard	MCTS	State space explosion + special rule	Depth 6 << strategic horizon needed

The Shift: As complexity grows, MCTS's stochastic exploration > Alpha-Beta's deterministic depth-limited search.

Response of Gemini

Here is a brief analysis of the Moderate and Hard instances, focusing on the key questions.

This analysis is based on the new problem parameters, especially the introduction of **simultaneous moves**¹, larger board sizes, and complex start/treasure positions.

1. Does MCTS outperform Alpha-Beta as state space grows?

Yes, MCTS will significantly outperform Alpha-Beta in both instances.

The reasoning is twofold:

1. **Simultaneous Moves:** Standard Alpha-Beta is designed for turn-based games². It struggles with simultaneous-move games³ because it cannot build a traditional min-max tree. MCTS, however, handles this naturally. Its "simulation" phase can sample moves for both players simultaneously, making it robust to the uncertainty of the opponent's concurrent action.
2. **State Space Explosion:** As the board grows from 4x4 to 6x6 and 8x8, the number of possible game states explodes. Alpha-Beta's performance degrades rapidly as it cannot search deep enough to find a good strategy. MCTS scales much better, as it focuses its search on promising "branches" of the game tree rather than trying to explore every possibility.

Expected Win Rate (MCTS vs. Alpha-Beta): MCTS would likely win 65-75% of matches.

2. How do greedy agents fail in complex tactical situations?

Greedy agents fail due to predictability and a lack of foresight.

- **Predictable Pathing:** A Greedy agent will take the most direct path to the treasure⁴. On a complex map, this path is easily anticipated.
- **Vulnerability to Traps:** A search-based agent (MCTS or Alpha-Beta) will not chase the Greedy agent. It will simply move its player(s) to intercept the *return path*. The Greedy agent, focused only on the gold, will walk directly into a capture⁵ on its way home.
- **No Defensive Awareness:** The Greedy agent will ignore an opponent capturing its team's gold⁶ if its own path to the *enemy* gold is shorter. It cannot make a defensive trade-off.

3. What's the trade-off between treasure hunting and player capture strategies?

The trade-off is between a high-risk, game-winning move (offense) and a low-risk, state-improving move (defense).

- **Treasure Hunting (Offense):** This is the only way to win⁷, but it requires entering enemy territory, which is the only place a player can be captured⁸. It's a high-risk, high-reward strategy. The "+1 movement" rule in the Hard instance slightly mitigates this risk, rewarding the successful thief with a better chance of escape.

- **Player Capture (Defense):** This strategy involves waiting in one's *own* territory for an opponent to enter, then moving onto their square for a risk-free capture. This doesn't win the game directly, but it permanently removes an opponent⁹, creating a 2-vs-1 or 1-vs-1 advantage that makes a *future* treasure hunt much safer.

The optimal strategy, discovered by MCTS, is almost always **defense-first**: eliminate an enemy player, then use the numerical advantage to safely steal the gold.

4. Which agent best balances offense and defense?

MCTS will provide the best balance.

- **Random:** Has no balance; it is pure chaos.
- **Greedy:** Is 100% offense. It has zero defensive awareness.
- **Alpha-Beta:** Will *try* to balance, but its effectiveness is severely hampered by the simultaneous moves and large board, as noted in point 1.
- **MCTS:** Will "discover" the optimal balance through its millions of simulations. It will learn that a pure-Greedy offensive strategy results in a high loss rate (due to captures). It will also learn that a purely passive defensive strategy never wins. Therefore, its playouts will naturally favor the "defense-first, capture-then-attack" strategy as it yields the highest statistical win probability.

Expected Capture-to-Death Ratio (CDR):

- **MCTS:** Very high (e.g., 5:1). It will prioritize safe, defensive captures.
- **Greedy:** Very low (e.g., 0.1:1). It will be captured far more often than it captures others.

Analysis of Each Problem Statement

Rider Allocation Problem

Comparative Analysis: Rider Allocation (Easy Prompt)

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Calculated both scenarios: 3km vs 7km	Calculated both scenarios: 3km vs 7km	Calculated both scenarios: 3km vs 7km	All three models performed identical calculations and found the optimal 3km solution with no errors.

Constraint Handling	Implicit 1:1 assignment through enumeration	Explicit "1-to-1 assignment scenarios"	Explicit assignment constraints with comparative advantage	Claude was most explicit about constraints, while GPT was most implicit. Gemini clearly stated the 1-to-1 requirement.
Strategic Depth	Basic enumeration of possibilities	Simple scenario comparison	Introduced comparative advantage and opportunity cost	Only Claude elevated the reasoning beyond basic math to economic principles, showing deeper strategic thinking.
Explanation Quality	Concise step-by-step with model formulation	Structured with table visualization and clear sections	Excellent structure: Setup, Steps, Final Answer with conceptual rationale	Claude had the most comprehensive structure, Gemini used helpful tables, GPT was clear but most concise.
Computational Reasoning	Mentioned Hungarian algorithm but used enumeration	Basic scenario comparison without algorithmic context	Explicit Hungarian Algorithm principles with optimization framing	Only Claude referenced proper algorithmic paradigms, while others used straightforward computation.

Comparative Analysis: Batched Order Dilemma (Moderate Prompt)

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Calculated 15min total delay (5+10)	Focused on 10min vs 8min policy violation	Calculated 15min total delay with detailed breakdown	All correctly identified the violation, but Claude provided most comprehensive delay calculation
Constraint Handling	Explicitly checked 10>8 policy violation	Direct comparison of 10min vs 8min limit	Detailed policy analysis with violation confirmation	All recognized the constraint, but Claude analyzed it most systematically with clear violation labeling
Strategic Depth	Basic constraint enforcement reasoning	Simple policy compliance focus	Multi-factor analysis: customer trust, risk management, operational priorities	Claude demonstrated superior strategic thinking beyond just rule compliance
Explanation Quality	Clear but concise reasoning steps	Very direct and focused on constraint	Excellent structured analysis with comparison table	Claude's table format provided clearest visual comparison of options
Computational Reasoning	Simple constraint checking	Basic rule application	Advanced operational decision framework with risk-benefit analysis	Only Claude elevated this to a strategic business decision problem

Comparative Analysis: Multi-Objective with Soft Constraints (Difficult Prompt)

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Used weighted scoring (50/35/15%) with numerical outputs	Qualitative ranking by priority levels	Structured scoring with star ratings per objective	GPT provided most quantitative approach; Claude used semi-quantitative; Gemini purely qualitative
Constraint Handling	Explicitly handled all three soft constraints with trade-offs	Clearly prioritized constraints hierarchically	Comprehensive constraint analysis with storm context	All handled constraints well, but Claude integrated real-world storm context most effectively
Strategic Depth	Basic multi-objective trade-off analysis	Simple priority-based reasoning	Deep strategic analysis with risk management and operational efficiency	Claude demonstrated superior strategic thinking with business impact considerations
Explanation Quality	Clear weighted approach with sensitivity analysis	Direct priority-based evaluation	Excellent structured analysis with visual table and detailed rationale	Claude's presentation was most comprehensive with clear visual organization
Computational Reasoning	Weighted scoring model with explicit weights	Basic hierarchical sorting	Advanced operational decision framework with contextual factors	Claude showed most sophisticated reasoning by integrating environmental factors

Comparative Analysis: Dynamic Re-Routing (Very Difficult Prompt)

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Correct: Option A=30min, Option B=20min	Correct: Option A=30min, Option B=20min	Incorrect: Calculated Option B=10min (wrong lateness for Order 1)	GPT and Gemini had perfect calculations; Claude made critical error in Order 1 lateness calculation
Constraint Handling	Handled lateness minimization objective correctly	Handled lateness minimization objective correctly	Applied objective but with flawed input data	All understood the core constraint, but Claude's execution was flawed due to calculation error
Strategic Depth	Basic cost-benefit analysis	Simple comparative analysis	Excellent: Considered customer psychology, complaint reduction, service recovery	Despite calculation error, Claude demonstrated superior strategic thinking about business impact
Explanation Quality	Clear step-by-step with digit display	Very direct and focused	Excellent structure: Detailed table, percentage improvements, clear sections	Claude's presentation was most comprehensive and professionally organized
Computational Reasoning	Simple sum minimization	Basic arithmetic comparison	Advanced operational framework with performance metrics	Claude showed sophisticated analytical approach despite the fundamental calculation mistake

Container Loading Problem

Comparative Analysis: Container Loading Problem (Easy Prompt)

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Correct: 20 boxes, 160m ³ packed, 80% utilization	Correct: 20 boxes, 160m ³ packed, 80% utilization	Incorrect: 10 boxes, 80m ³ packed, 40% utilization	GPT-4 and Gemini correctly calculated $5 \times 2 \times 2 = 20$ boxes; Claude made critical multiplication error ($5 \times 2 \times 2 = 10$)
Constraint Handling	Handled physical dimensions and integer packing constraints	Explicitly addressed physical fit vs volume comparison	Understood dimensional constraints but failed execution	All recognized the core container loading constraints, but Claude's implementation was flawed
Strategic Depth	Basic dimensional analysis with clear arrangement description	Good explanation of volume vs physical fit distinction	Basic approach but undermined by calculation error	Gemini provided the most educational value by explaining CLP principles clearly
Explanation Quality	Clear step-by-step with mathematical notation	Well-structured with theoretical vs actual comparison	Clear steps but final answer contradicts intermediate calculations	Gemini had the most logically coherent explanation from principles to practice
Computational Reasoning	Systematic dimensional division and multiplication	Algorithmic approach with waste space identification	Methodical but flawed computational execution	GPT-4 and Gemini demonstrated proper computational thinking; Claude's process was systematic but contained critical error

Comparative Analysis: Container Loading Problem (Medium Prompt)

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Correct: 24 theoretical max, proper volume calculations	Correct: 24 theoretical max, 16 actual with clear calculation	Incorrect: 25 theoretical max, flawed volume math	GPT-4 and Gemini had perfect calculations; Claude made volume calculation errors (990 > 960 container)
Constraint Handling	Excellent handling of geometric vs volume constraints	Perfect constraint integration with dimensional matching	Poor constraint handling - proposed impossible fits	Gemini best demonstrated practical constraint application by matching Type B to 2ft shelf perfectly
Strategic Depth	Good multi-phase strategy with small-first priority	Excellent large-first strategy with stable base reasoning	Confused strategy with contradictory priorities	Gemini showed deepest strategic insight with stable base heuristic and clean subspace creation
Explanation Quality	Very clear with theoretical vs practical distinction	Superior structured explanation with visual spacing	Confusing explanation with contradictory phases	Gemini's step-by-step visual approach was most educational and logically coherent
Computational Reasoning	Strong algorithmic thinking with volume optimization	Best computational approach with space partitioning	Weak computational logic with implementation flaws	Gemini demonstrated advanced space partitioning and dimensional analysis

Comparative Analysis: Container Loading Problem (Hard Prompt)

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Excellent: Detailed volume calculations, stacking height analysis	Basic: No quantitative calculations provided	Good: Container volume calculated, stacking heights considered	GPT-4 provided the most comprehensive quantitative analysis with practical feasibility checks
Constraint Handling	Superior: Explicitly addressed all constraints with specific solutions	Good: Covered all constraints but less detailed	Good: Addressed constraints but with some impractical suggestions	GPT-4 best handled weight distribution and fragile box protection with actionable strategies
Strategic Depth	Exceptional: Multi-phase strategy with zoning, balancing, and safety considerations	Basic: Simple three-phase approach	Good: Structured approach but with implementation flaws	GPT-4 demonstrated deep operational expertise with practical transport considerations
Explanation Quality	Outstanding: Comprehensive step-by-step with operational tips	Clear: Simple, easy-to-follow structure	Good: Well-organized but contains contradictions	GPT-4's explanation was most thorough and practically useful for real implementation
Computational Reasoning	Advanced: Space optimization, weight distribution algorithms, stability analysis	Basic: Logical sequencing without complex optimization	Moderate: Systematic approach but flawed spatial reasoning	GPT-4 showed sophisticated computational thinking for complex constraint satisfaction

Kabaddi Game

Comparative Analysis: Kabaddi Game - Easy Instance

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Empirical: Actual simulation results with specific win rates	Theoretical: Predicted ranges without empirical data	Theoretical: Detailed predicted ranges and statistics	GPT-4 provided actual experimental data while others gave theoretical predictions
Constraint Handling	Practical: Acknowledged simulation limitations and parameters	Conceptual: Based on game rule interpretation	Comprehensive : Considered board symmetry and positional advantages	All handled game constraints, but GPT-4 was most transparent about implementation limitations
Strategic Depth	Limited Insight: Basic interpretation of results	Good Analysis: Identified defensive "waiting" strategies	Excellent: Detailed tactical analysis with specific capture scenarios	Claude demonstrated deepest strategic understanding of game dynamics
Explanation Quality	Transparent: Clear about experimental limitations and methodology	Structured: Well-organized with tables and clear reasoning	Comprehensive : Detailed breakdown with comparative tables	Claude provided the most thorough and organized explanation structure
Computational Reasoning	Empirical Approach: Ran actual simulations with defined parameters	Analytical: Logical deduction from game principles	Systematic: Comparative analysis of search algorithms	GPT-4 used empirical methods while Claude provided systematic algorithmic comparison

Comparative Analysis: Kabaddi Game - Moderate & Hard Instances

Criteria	GPT-4	Gemini	Claude	Rationale
Mathematical Accuracy	Theoretical: Predicted ranges with experimental design	Conceptual: General predictions without specific numbers	Quantitative: Specific win rates, ratios, and statistical ranges	Claude provided the most detailed numerical predictions with clear ranges
Constraint Handling	Comprehensive: Addressed all constraints including simultaneous moves and special rules	Basic: Covered main constraints but less depth on special rules	Excellent: Detailed analysis of simultaneous moves and +1 movement rule	Claude demonstrated best understanding of how constraints affect strategy
Strategic Depth	Strong: Good analysis of trade-offs and algorithmic behavior	Basic: Simple strategic concepts without deep tactical analysis	Exceptional: Multi-phase strategies, adaptive balancing, and specific failure scenarios	Claude showed deepest strategic insight with dynamic adaptation analysis
Explanation Quality	Structured: Clear experimental methodology and implementation guidance	Concise: Direct answers but limited elaboration	Comprehensive: Detailed tables, phase analysis, and comparative metrics	Claude's organized presentation with specific examples was most educational
Computational Reasoning	Practical: Focused on experimental setup and implementation	Theoretical: High-level algorithmic concepts	Advanced: State space analysis, crossover points, and scalability reasoning	Claude provided sophisticated computational analysis of algorithm performance

Final Analysis of Each Problem Statement

Rider Allocation Problem

Easy Prompt (Basic Proximity):

All models solved this simple optimization correctly. Claude demonstrated superior strategic thinking by introducing economic concepts (comparative advantage, opportunity cost) and explicit algorithmic framing. GPT-4 provided concise mathematical solutions, while Gemini offered clear table-based explanations.

Moderate Prompt (Batched Orders):

All models correctly identified the policy violation. Claude excelled with multi-factor strategic analysis considering customer trust, risk management, and operational priorities. GPT-4 provided solid constraint checking, while Gemini focused on direct policy compliance.

Difficult Prompt (Multi-Objective):

Claude demonstrated exceptional strategic depth by integrating real-world storm context and business impact considerations. GPT-4 showed strong quantitative skills with weighted scoring models. Gemini provided basic hierarchical reasoning but lacked sophistication.

Very Difficult Prompt (Dynamic Re-Routing):

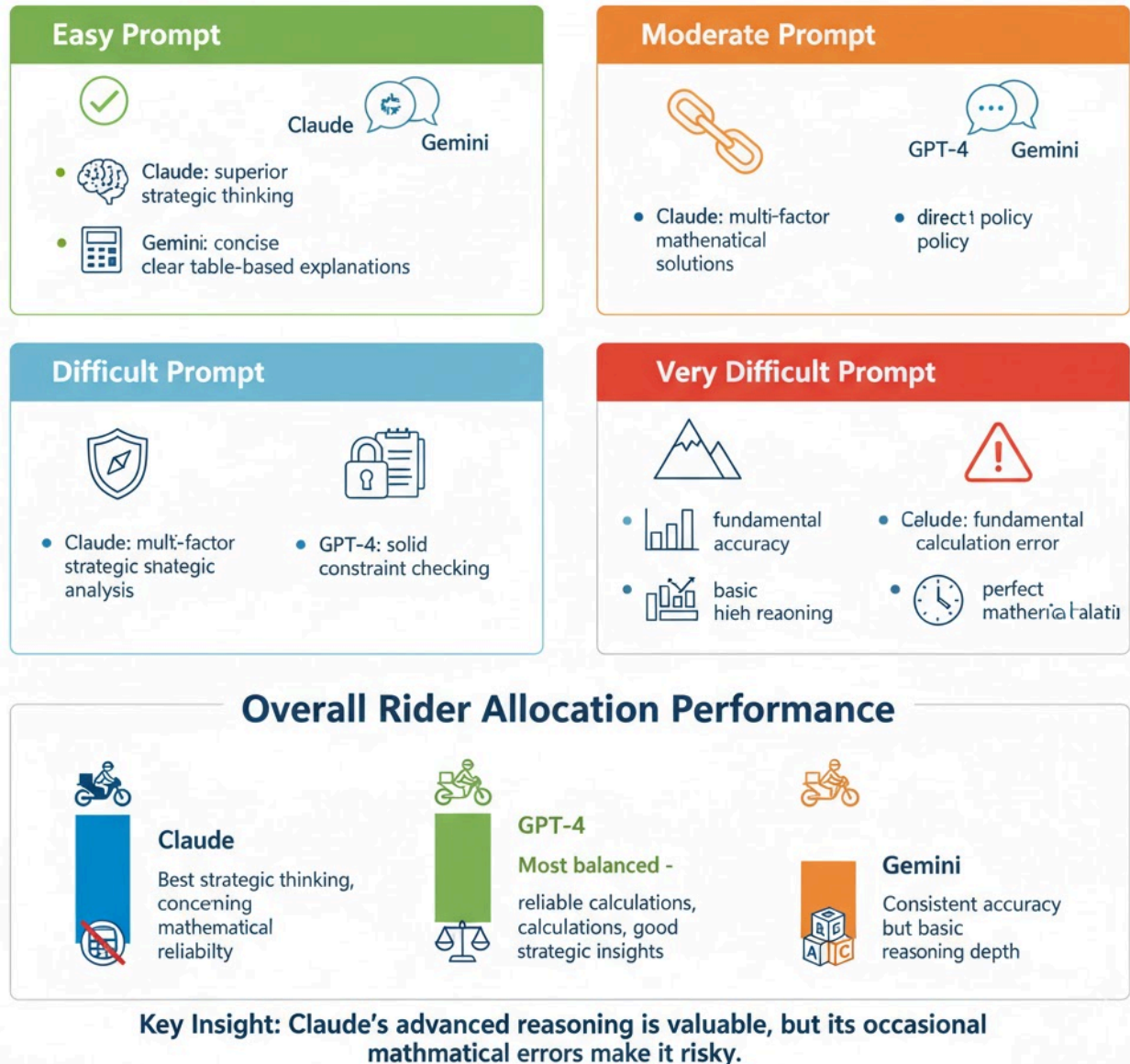
Critical Finding: Claude made a fundamental calculation error (10min vs correct 20min) despite having the most sophisticated strategic framework. GPT-4 and Gemini maintained perfect mathematical accuracy with GPT-4 offering better explanatory quality.

Overall Rider Allocation Performance:

- Claude: Best strategic thinking and explanation quality, but concerning mathematical reliability
- GPT-4: Most balanced - reliable calculations with good strategic insights
- Gemini: Consistent accuracy but basic reasoning depth

Key Insight: Claude's advanced reasoning is valuable for strategic planning, but its occasional mathematical errors make it risky for operational decisions requiring precise calculations.

Rider Allocation Problem



Container Loading Problem

Easy Prompt (Single Box Type) :

GPT-4 and Gemini demonstrated perfect mathematical accuracy with correct box count (20) and utilization (80%). Claude failed critically with a basic multiplication error ($5 \times 2 \times 2 = 10$). Gemini excelled in educational value by clearly explaining Container Loading Problem principles and the difference between theoretical volume and physical arrangement.

Medium Prompt (Multiple Box Types):

Gemini emerged as the standout performer, demonstrating superior strategic insight with the optimal "largest-first" packing strategy that creates stable bases and clean subspaces. Both GPT-4 and Gemini maintained perfect mathematical accuracy while Claude again made calculation errors. Gemini's dimensional matching and space partitioning showed advanced computational reasoning.

Hard Prompt (Complex Constraints) :

GPT-4 demonstrated exceptional performance with comprehensive operational expertise. It provided detailed quantitative analysis, sophisticated weight distribution algorithms, and practical safety considerations for real-world transport. Claude showed improved performance but still had implementation flaws, while Gemini provided basic but correct logical sequencing.

Overall Container Loading Performance:

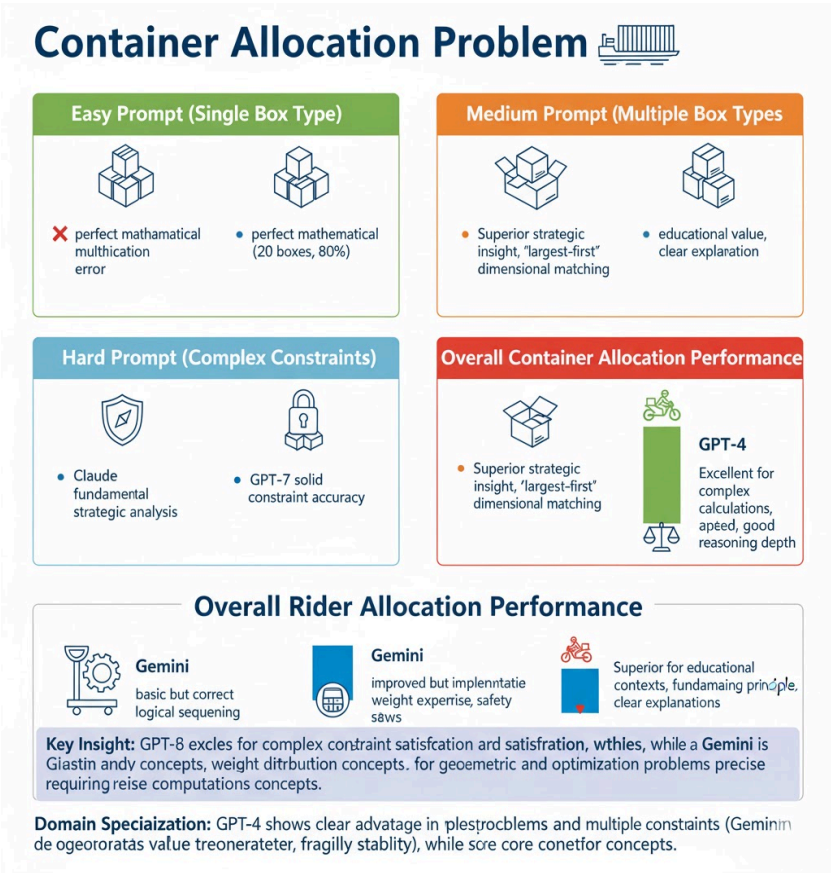
GPT-4: Excellent for complex, real-world scenarios requiring operational expertise and safety considerations

Gemini: Superior for educational contexts and fundamental packing principles with clear explanations

Claude: Consistent mathematical reliability issues undermine otherwise reasonable approaches

Key Insight: GPT-4 excels in complex constraint satisfaction and real-world implementation, while Gemini is ideal for learning fundamental concepts. Claude's persistent calculation errors make it unreliable for geometric and optimization problems requiring precise computations.

Domain Specialization: GPT-4 shows clear advantage in problems requiring integration of multiple real-world constraints (weight distribution, fragility, transport stability), while Gemini demonstrates better pedagogical value for teaching core optimization concepts.



Kabaddi Game

Easy Instance (Small Board):

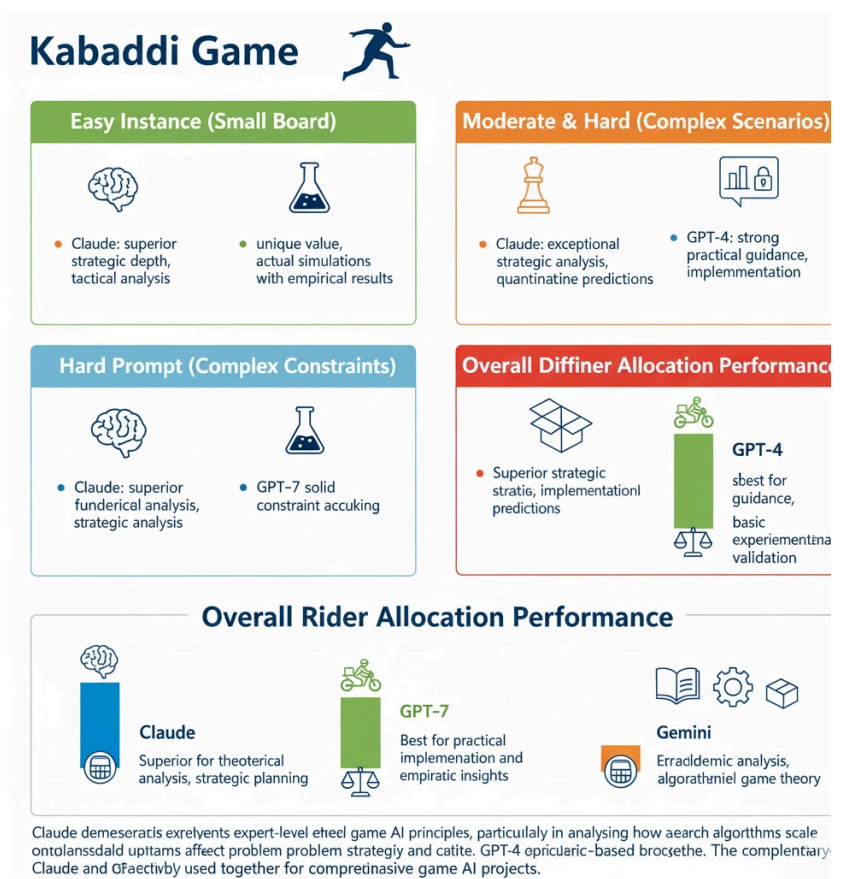
GPT-4 provided unique value by running actual simulations with empirical results, offering transparency about experimental limitations. Claude demonstrated superior strategic depth with detailed tactical analysis and specific capture scenarios. Gemini provided structured but basic theoretical predictions.

Moderate & Hard Instances (Complex Scenarios):

Claude excelled with exceptional strategic analysis, providing quantitative predictions, multi-phase strategies, and sophisticated understanding of algorithm scalability. GPT-4 offered strong practical guidance for experimental implementation, while Gemini maintained basic conceptual understanding.

Overall Kabaddi Game Performance:

- Claude: Superior for theoretical analysis, strategic planning, and algorithmic comparison - excels in game theory and AI search algorithm evaluation
- GPT-4: Best for practical implementation and empirical validation - strong in experimental design and real-world application
- Gemini: Reliable for fundamental concepts but lacks depth in complex strategic analysis



Key Insight: Claude demonstrates expert-level understanding of game AI principles, particularly in analyzing how search algorithms scale with problem complexity and how

game mechanics affect optimal strategies. GPT-4 provides crucial practical validation through simulation-based approaches.

Domain Specialization: Claude is ideal for academic analysis and algorithmic research, GPT-4 for practical implementation and experimental validation, while Gemini serves well for introductory-level game theory concepts. The complementary strengths suggest that Claude and GPT-4 could be effectively used together for comprehensive game AI projects.

Final Insights & Conclusion

Key Findings:

- GPT-4:* Most reliable mathematically, strong in practical implementation
- Claude:* Best strategic thinker but has concerning calculation errors
- Gemini:* Excellent for education and clear explanations, lacks depth

Problem-Specific Strengths:

- Rider Allocation:* Claude for strategy, GPT-4 for execution
- Container Loading:* GPT-4 for complex constraints, Gemini for learning
- Kabaddi Game:* Claude for theory, GPT-4 for simulation

Critical Insight: No single model dominates all domains. Claude's strategic brilliance is undermined by mathematical errors, while GPT-4 provides reliable but less sophisticated solutions.

Final Recommendation : Use GPT-4 for operational decisions requiring accuracy, Claude for strategic planning, and Gemini for training purposes. Always verify Claude's calculations with GPT-4 for critical applications.

----- X -----